AWS

CHEAT SHEETS

Amazon Web Services for developers and operators, from the AWS CLI to Bedrock

AWS CLI · Organizations · IAM · STS & Roles · OIDC · Secrets Manager · KMS · GuardDuty · VPC · Security Groups · Route 53 · CloudFront · EC2 · Auto Scaling · ALB · Lambda · API Gateway · EventBridge · SQS / SNS · ECR · ECS / Fargate · EKS · S3 · RDS / Aurora · DynamoDB · CloudFormation · CDK · CloudWatch · Bedrock

42 REFERENCE SHEETS

CONTENTS

GETTING STARTED

01 Overview & Global Infrastructure
02 Account Setup & Billing
03 AWS CLI v2
04 SDKs & Credentials

IDENTITY & SECURITY

05 IAM Users, Groups & Roles
06 IAM Policies
07 Roles & Temporary Credentials
08 Secrets, Parameters & KMS
09 Security & Audit Services

NETWORKING

10 VPC Basics
11 NAT, Endpoints & Connectivity
12 Security Groups & NACLs
13 Route 53
14 CloudFront & ACM

COMPUTE

15 EC2 Basics
16 EC2 Operations & Pricing
17 Elastic Load Balancing

SERVERLESS & MESSAGING

20 Lambda Basics
21 Lambda Operations
22 API Gateway
23 EventBridge & Step Functions
24 SQS & SNS

CONTAINERS

25 Elastic Container Registry
26 ECS & Fargate
27 EKS

STORAGE

28 S3 Basics
29 S3 Security & Features
30 EBS, EFS & AWS Backup

DATABASES

31 RDS & Aurora
32 DynamoDB
33 ElastiCache & Other Databases

INFRASTRUCTURE AS CODE

34 CloudFormation
35 AWS CDK
36 SAM & Terraform on AWS

OBSERVABILITY & DEPLOYMENT

37 CloudWatch Metrics & Alarms
38 CloudWatch Logs & Tracing
39 Deploying to AWS

AI & MACHINE LEARNING

40 Amazon Bedrock
41 AI & ML Services

QUICK REFERENCE

42 Quick Reference — CLI & Service Picker

AWS

Table of Contents

42 sheets · syntax · examples · notes

GETTING STARTED 01 – 04

- 01 Overview & Global Infr…
- 02 Account Setup & Billing
- 03 AWS CLI v2
- 04 SDKs & Credentials

IDENTITY & SECURITY 05 – 09

- 05 IAM Users, Groups & R…
- 06 IAM Policies
- 07 Roles & Temporary Cre…
- 08 Secrets, Parameters & …
- 09 Security & Audit Servi…

NETWORKING 10 – 14

- 10 VPC Basics
- 11 NAT, Endpoints & Conn…
- 12 Security Groups & NA…
- 13 Route 53
- 14 CloudFront & ACM

COMPUTE 15 – 19

- 15 EC2 Basics
- 16 EC2 Operations & Prici…
- 17 Elastic Load Balancing
- 18 Auto Scaling
- 19 Choosing Compute

SERVERLESS & MESSAGING 20 – 24

- 20 Lambda Basics
- 21 Lambda Operations
- 22 API Gateway
- 23 EventBridge & Step Fu…
- 24 SQS & SNS

CONTAINERS 25 – 27

- 25 Elastic Container Regis…
- 26 ECS & Fargate
- 27 EKS

STORAGE 28 – 30

- 28 S3 Basics
- 29 S3 Security & Features
- 30 EBS, EFS & AWS Backup

DATABASES 31 – 33

- 31 RDS & Aurora
- 32 DynamoDB
- 33 ElastiCache & Other D…

INFRASTRUCTURE AS CODE 34 – 36

- 34 CloudFormation
- 35 AWS CDK
- 36 SAM & Terraform on A…

OBSERVABILITY & DEPLOYMENT 37 – 39

- 37 CloudWatch Metrics &…
- 38 CloudWatch Logs & Tr…
- 39 Deploying to AWS

AI & MACHINE LEARNING 40 – 41

- 40 Amazon Bedrock
- 41 AI & ML Services

QUICK REFERENCE 42

- 42 Quick Reference — CLI…

Sheet 01

Overview & Global Infrastructure

Regions · Availability Zones · edge locations · global vs regional services · shared responsibility model · Free Tier · ARNs

AMAZON WEB SERVICES

AWS runs in 30+ **Regions**, each with three or more isolated **Availability Zones** (AZs) and hundreds of **edge locations** for CloudFront and Route 53. Most services are regional (EC2, S3 buckets, Lambda); a few are global (IAM, Route 53, CloudFront). Every resource is named by an **ARN**, and the **account** is the main security and billing boundary.

KEY POINTS

- Region codes: `us-east-1`, `us-west-2`, `eu-west-1`
- AZ names (`us-east-1a`) map to different buildings per account; AZ IDs (`use1-az1`) are fixed
- Some Regions are opt-in and must be enabled first
- Shared responsibility: AWS secures the cloud, you secure what you put in it

EXPLORE REGIONS, AZS AND ARNS

```
aws sts get-caller-identity                    # account and principal
aws ec2 describe-regions --query "Regions[].RegionName" --output text
aws ec2 describe-availability-zones --region us-east-1 \
    --query "AvailabilityZones[].[ZoneName,ZoneId]" --output table
aws account list-regions --region-opt-status-contains ENABLED_BY_DEFAULT

# ARN format: arn:partition:service:region:account-id:resource
# arn:aws:s3:::acme-logs                       (S3: no region / account)
# arn:aws:lambda:us-east-1:111122223333:function:resize
# arn:aws:iam::111122223333:role/app-role      (IAM: global, no region)

aws resourcegroupstaggingapi get-resources \
    --tag-filters Key=env,Values=prod \
    --query "ResourceTagMappingList[].ResourceARN"
```

CODE EXPLAINED

- → `get-caller-identity` shows who and where you are
- → Zone IDs let you line up AZs across accounts
- → ARNs leave out region or account when a service is global
- → The tagging API lists resources across many services by tag

AZURE / GCP EQUIVALENT

- ↔ **Azure** regions, zones, region pairs, resource groups (az-01)
- ↔ **GCP** regions, zones, global VPCs, projects (gcp-01)
- ↔ AWS uses the account where Azure uses subscription + resource group

COST CHECK

- $ Free Tier: credits for new accounts plus always-free allowances (Lambda requests, DynamoDB storage)
- $ Prices vary by Region; us-east-1 is often cheapest
- $ Inter-AZ and inter-Region data transfer is billed per GB

SCOPE

SCOPE	EXAMPLES
Global	IAM, Route 53, CloudFront, Organizations
Regional	S3 buckets, Lambda, DynamoDB, VPC
Zonal	EC2 instance, EBS volume, subnet
Edge	CloudFront, Global Accelerator, Route 53

WAYS TO WORK

TOOL	USE
Console	Explore, one-offs
AWS CLI v2	Scripts, this set
CloudShell	Browser shell
SDKs (boto3, JS v3)	Apps
CloudFormation / CDK	Repeatable infra

DO & DON'T

- ✓ **Do** Pick a home Region and keep resources there unless there is a reason
- ✓ **Do** Tag everything (owner, env, app)
- ✗ **Don't** Use the root user for anything but account-level tasks
- ✗ **Don't** Forget global services when cleaning up a Region

LOCAL TESTING

- ⌂ CloudShell has the CLI preinstalled and authenticated
- ⌂ LocalStack emulates many AWS APIs locally for development
- ⌂ A new account with a budget alert is a safe sandbox

PRODUCTION CHECKLIST

- ☐ Multi-AZ by default for production services
- ☐ Separate accounts per environment (Organizations)
- ☐ Tag policy and cost allocation tags enabled

Account Setup & Billing

SETTING UP AN ACCOUNT SAFELY

A new account starts with a **root user** that can do anything: lock it with MFA and stop using it. Create an **Organization** to group accounts into organizational units (OUs) with consolidated billing and **service control policies (SCPs)**. People sign in through **IAM Identity Center** (SSO). **Budgets** alert on spend and **Cost Explorer** shows where it goes.

KEY POINTS

- Root user only for tasks that require it (account closure, some billing settings)
- Landing zones: Control Tower sets up accounts, logging and guardrails
- Cost allocation tags must be activated in Billing to appear in reports
- Budgets can trigger actions (apply an SCP or IAM policy)

LIMITS & QUOTAS

- ≤ New organizations start with a small default account quota (raise in Service Quotas)
- ≤ SCP document size up to 5,120 characters; 5 SCPs per OU / account
- ≤ 50 user tags per resource

ORGANIZATION, SSO, BUDGETS

```
aws organizations create-organization --feature-set ALL
aws organizations create-organizational-unit --parent-id r-ab12 --name Prod
aws organizations create-account --email shop-prod@example.com \
    --account-name shop-prod
aws organizations list-accounts --output table

aws budgets create-budget --account-id 111122223333 \
    --budget file://budget.json \
    --notifications-with-subscribers file://notify.json
# budget.json: {"BudgetName": "monthly", "BudgetType": "COST",
#   "TimeUnit": "MONTHLY", "BudgetLimit": {"Amount": "300", "Unit": "USD"}}
# notify.json: [{"Notification": {"NotificationType": "ACTUAL",
#   "ComparisonOperator": "GREATER_THAN", "Threshold": 80},
#   "Subscribers": [{"SubscriptionType": "EMAIL", "Address": "ops@acme.com"}]}]

aws ce get-cost-and-usage --time-period Start=2026-10-01,End=2026-11-01 \
    --granularity MONTHLY --metrics UnblendedCost \
    --group-by Type=TAG,Key=team

aws service-quotas list-service-quotas --service-code organizations
```

CODE EXPLAINED

- → `feature-set ALL` enables SCPs and other policy types
- → New accounts are created inside the organization with their own root
- → The budget emails at 80% of actual monthly spend
- → Cost Explorer groups spend by the activated `team` tag

AZURE / GCP EQUIVALENT

- ↔ **Azure** management groups, subscriptions, Cost Management budgets (az-02)
- ↔ **GCP** organization, folders, projects, billing budgets (gcp-02)

COST CHECK

- $ Organizations, SCPs and Identity Center are free
- $ The first two budgets are free; more cost a little per day
- $ Cost Explorer UI is free; its API bills per request

ACCOUNT STRUCTURE

PIECE	ROLE
Management account	Billing, org admin only
OU	Group accounts, attach SCPs
Member accounts	Workloads per env / team
Log archive account	Central CloudTrail / Config
Security account	GuardDuty / Security Hub admin

COST TOOLS

TOOL	DOES
Budgets	Alerts and actions
Cost Explorer	Explore spend
Cost Anomaly Detection	Unusual spend alerts
Data Exports (CUR 2.0)	Detailed billing data
Compute Optimizer	Rightsizing

DO & DON'T

- ✓ **Do** Enable MFA on root and store its credentials offline
- ✓ **Do** Use Identity Center for all human access
- ✗ **Don't** Run workloads in the management account
- ✗ **Don't** Create IAM users with long-lived keys for people

LOCAL TESTING

- ⌂ Practice in a sandbox account inside your organization
- ⌂ Price designs with the AWS Pricing Calculator
- ⌂ Infracost estimates CloudFormation / Terraform changes

PRODUCTION CHECKLIST

- ☐ Root MFA, alternate contacts and billing alerts set
- ☐ SCP guardrails on OUs (deny leaving org, disabling CloudTrail)
- ☐ Cost allocation tags activated and enforced

AWS CLI v2

SCRIPTING AWS

The **AWS CLI v2** calls every AWS API from the shell: `aws <service> <operation>`. Sign in with **IAM Identity Center** via `aws configure sso`, which stores a profile that refreshes short-lived credentials. `--query` filters output with JMESPath, `--output` picks json, table, text or yaml, and many services add high-level commands like `aws s3`.

KEY POINTS

- Config lives in `~/.aws/config` (profiles) and `~/.aws/credentials` (keys, avoid)
- `AWS_PROFILE` and `AWS_REGION` environment variables override defaults
- Pagination is automatic; `--max-items` limits it
- `--dry-run` checks EC2 permissions without acting

SECURITY CHECK

- ⚠ Prefer SSO profiles over `aws configure` access keys
- ⚠ Never commit `~/.aws/credentials` or paste keys into scripts
- ⚠ Use `--profile` explicitly in scripts that touch production

EVERYDAY AWS

```
aws configure sso                          # one-time: creates a profile
aws sso login --profile shop-dev
export AWS_PROFILE=shop-dev AWS_REGION=us-east-1

aws sts get-caller-identity
aws ec2 describe-instances \
    --filters Name=instance-state-name,Values=running \
    --query "Reservations[].Instances[].[InstanceId,InstanceType]" \
    --output table

aws s3 ls
aws lambda list-functions --query "Functions[].FunctionName" --output text
aws ec2 run-instances --dry-run --image-id ami-0abc --instance-type t4g.micro

ID=$(aws ec2 describe-vpcs --filters Name=isDefault,Values=true \
     --query "Vpcs[0].VpcId" --output text)
aws cloudformation deploy help             # built-in docs
aws --cli-auto-prompt                      # interactive completion
```

CODE EXPLAINED

- → `configure sso` asks for the SSO start URL, account and role
- → JMESPath flattens reservations into instance ID and type
- → `--output text` makes values easy to capture in variables
- → `--dry-run` answers "would I be allowed?"

AZURE / GCP EQUIVALENT

- ↔ **Azure** Azure CLI, `--query`, `az account set` (az-03)
- ↔ **GCP** gcloud, configurations, `--format` / `--filter` (gcp-03)

COST CHECK

- $ The CLI is free; every call to a billable API costs the same as from the console
- $ CloudShell is free (1 GB persistent storage per Region)
- $ Watch loops that call paid APIs (e.g. Cost Explorer) repeatedly

GLOBAL OPTIONS

OPTION	DOES
`--profile p`	Pick profile
`--region r`	Pick Region
`--output json|table|text|yaml`	Format
`--query`	JMESPath filter
`--no-cli-pager`	Disable pager
`--debug`	Show HTTP calls

CONFIG FILES

FILE	HOLDS
`~/.aws/config`	Profiles, SSO, Region
`~/.aws/credentials`	Static keys (avoid)
`~/.aws/sso/cache`	SSO tokens
`AWS_PROFILE / AWS_REGION`	Env overrides

DO & DON'T

- ✓ **Do** Use one named SSO profile per account / role
- ✓ **Do** Set `--no-cli-pager` or `AWS_PAGER=""` in scripts
- ✗ **Don't** Create access keys for your own laptop use
- ✗ **Don't** Parse table output in scripts

LOCAL TESTING

- ⌂ `aws --endpoint-url http://localhost:4566 s3 ls` talks to LocalStack
- ⌂ `awslocal` wraps the CLI for LocalStack
- ⌂ Docker image `amazon/aws-cli` runs the CLI without installing

PRODUCTION CHECKLIST

- ☐ Automation uses roles (OIDC / instance profiles), not user keys
- ☐ Scripts pin profile and Region explicitly
- ☐ CloudTrail records every CLI call for audit

SDKs & Credentials

credential provider chain · ~/.aws/config & credentials · AWS_PROFILE env vars · boto3 Session / client · JS SDK v3 clients · roles over access keys

AUTHENTICATING CODE

AWS SDKs look for credentials in a **provider chain**: environment variables, the shared config / SSO profile, web identity tokens (EKS, CI), the container credentials endpoint (ECS), then the EC2 instance metadata service. The same code uses your SSO profile on a laptop and the attached **IAM role** in AWS. Create clients once and reuse them.

KEY POINTS

- Python: `pip install boto3` ; JavaScript: `@aws-sdk/client-s3` (modular v3)
- Region comes from `AWS_REGION` , the profile or the client argument
- Retries with backoff are built in; tune with `retries` config
- Never hard-code keys in code or images

SECURITY CHECK

- ⚠ Use IAM roles for compute; no access keys on servers or in containers
- ⚠ Require IMDSv2 so SSRF bugs cannot steal instance credentials
- ⚠ Rotate or remove any remaining access keys (alert on keys older than 90 days)

PYTHON (BOTO3)

```python
import boto3
s3 = boto3.client("s3")                       # default chain + Region
for b in s3.list_buckets()["Buckets"]:
    print(b["Name"])

dev = boto3.Session(profile_name="shop-dev", region_name="us-east-1")
ddb = dev.resource("dynamodb").Table("orders")
ddb.put_item(Item={"pk": "o#42", "total": 59})

sts = boto3.client("sts")                     # assume another role
creds = sts.assume_role(RoleArn="arn:aws:iam::222233334444:role/reader",
                        RoleSessionName="report")["Credentials"]
```

CODE EXPLAINED

- → `boto3.client` finds credentials and Region automatically
- → A Session pins a profile for multi-account scripts
- → `assume_role` returns temporary keys for another account
- → JS v3 imports one client per service and sends command objects

JAVASCRIPT SDK V3

```javascript
import { S3Client, ListObjectsV2Command } from "@aws-sdk/client-s3";
const s3 = new S3Client({ region: "us-east-1" });   // default chain
const out = await s3.send(new ListObjectsV2Command({ Bucket: "acme-logs" }));
console.log(out.Contents?.map(o => o.Key));
```

AZURE / GCP EQUIVALENT

- ↔ **Azure** DefaultAzureCredential, managed identity (az-04)
- ↔ **GCP** Application Default Credentials, service accounts (gcp-04)

COST CHECK

- $ SDKs are free; you pay for the API calls they make
- $ STS calls are free
- $ Reusing clients and connections cuts latency and request overhead

CREDENTIAL CHAIN (SIMPLIFIED)

ORDER	SOURCE	WHERE
1	Environment variables	AWS_ACCESS_KEY_ID…
2	Shared config / SSO	Laptop profiles
3	Web identity token	EKS IRSA, CI OIDC
4	Container credentials	ECS tasks, EKS Pod Identity
5	Instance metadata	EC2 instance profile

PICK A METHOD

WHERE	USE
Laptop	SSO profile
EC2	Instance profile role
ECS / Fargate	Task role
Lambda	Execution role
EKS	Pod Identity / IRSA
GitHub Actions	OIDC role

DO & DON'T

- ✓ **Do** Create SDK clients once per process
- ✓ **Do** Let roles provide credentials everywhere in AWS
- ✗ **Don't** Put access keys in environment variables on servers
- ✗ **Don't** Disable SSL verification to "fix" endpoint errors

LOCAL TESTING

- ⌂ SSO profiles feed the same default chain on your laptop
- ⌂ LocalStack: set `endpoint_url="http://localhost:4566"` in the client
- ⌂ moto mocks AWS APIs inside Python unit tests

PRODUCTION CHECKLIST

- ☐ Zero long-lived access keys for workloads
- ☐ IMDSv2 required on all instances
- ☐ Least-privilege roles per workload

IAM Users, Groups & Roles

IDENTITIES IN AWS

IAM controls who can call AWS APIs in an account. **Users** have long-term credentials and are now mainly for edge cases; people should use **IAM Identity Center**. **Roles** have no long-term credentials: a trusted principal (a person, a service such as Lambda, another account, or an OIDC identity) **assumes** the role and gets temporary keys. **Groups** attach policies to many users.

KEY POINTS

- Every role has a trust policy (who can assume) and permission policies (what it can do)
- Service roles let AWS services act for you (Lambda execution role)
- Service-linked roles are created and managed by a service
- IAM is global within an account

CREATE A ROLE FOR LAMBDA

```
cat > trust.json <<'JSON'
{ "Version": "2012-10-17",
  "Statement": [{ "Effect": "Allow",
    "Principal": { "Service": "lambda.amazonaws.com" },
    "Action": "sts:AssumeRole" }] }
JSON
aws iam create-role --role-name resize-fn-role \
    --assume-role-policy-document file://trust.json
aws iam attach-role-policy --role-name resize-fn-role \
    --policy-arn \
    arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole

aws iam list-roles --query "Roles[?starts_with(RoleName,'resize')].Arn"
aws iam list-attached-role-policies --role-name resize-fn-role

# Who am I right now?
aws sts get-caller-identity
# Find old access keys
aws iam generate-credential-report && aws iam get-credential-report \
    --query Content --output text | base64 -d | cut -d, -f1,9,10
```

CODE EXPLAINED

- → The trust policy lets the Lambda service assume the role
- → The managed policy grants CloudWatch Logs access
- → JMESPath filters roles by name prefix
- → The credential report lists users and key ages

AZURE / GCP EQUIVALENT

↔	**Azure**	Entra ID users, groups, service principals (az-05)
↔	**GCP**	Google accounts, groups, service accounts (gcp-05)

COST CHECK

- $ IAM is free
- $ Leaked access keys are the most expensive mistake: crypto-mining bills
- $ Unused roles cost nothing but widen the attack surface

IDENTITY TYPES

TYPE	CREDENTIALS	USE
Root user	Password + MFA	Account-level tasks only
Identity Center user	SSO, temporary	People
IAM user	Password / access keys	Legacy, break-glass
IAM role	Temporary (STS)	Services, apps, cross-account
Group	None	Attach policies to users

DO & DON'T

- ✓ **Do** Use roles for every workload and Identity Center for people
- ✓ **Do** Remove unused users, keys and roles
- ✗ **Don't** Share IAM user credentials among a team
- ✗ **Don't** Attach AdministratorAccess to application roles

LOCAL TESTING

- ⌂ IAM Policy Simulator tests what a role can do
- ⌂ IAM Access Analyzer validates policies and finds unused access
- ⌂ LocalStack accepts IAM calls (enforcement optional)

PRODUCTION CHECKLIST

- ☐ No IAM users for humans; MFA on any that remain
- ☐ Credential report reviewed; keys rotated or removed
- ☐ Access Analyzer unused-access findings acted on

IAM Policies

`Effect` / `Action` / `Resource` / `Condition` · managed vs inline · identity vs resource policies · explicit deny wins · policy simulator · permission boundaries

WRITING PERMISSIONS

An IAM **policy** is a JSON document of statements with **Effect** (Allow / Deny), **Action** (`s3:GetObject`), **Resource** (ARNs) and optional **Condition** (IP, tags, MFA, time). Identity policies attach to users, groups and roles; resource policies attach to buckets, queues, keys and more. Requests are denied by default, allowed by any Allow, and blocked by any explicit **Deny**.

KEY POINTS

- AWS managed policies are broad; customer managed policies fit your needs
- Permission boundaries cap what a role can ever be granted
- SCPs (Organizations) cap entire accounts
- Use tags in conditions for attribute-based access control (ABAC)

LIMITS & QUOTAS

- ≤ Managed policy up to 6,144 characters
- ≤ 10 managed policies per role by default (can raise)
- ≤ Role trust policy up to 2,048 characters by default

A LEAST-PRIVILEGE POLICY

```
{ "Version": "2012-10-17",
  "Statement": [
    { "Sid": "ReadUploads",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": "arn:aws:s3:::acme-uploads/user/${aws:PrincipalTag/uid}/*" },
    { "Sid": "TeamTables",
      "Effect": "Allow",
      "Action": ["dynamodb:GetItem", "dynamodb:Query", "dynamodb:PutItem"],
      "Resource": "arn:aws:dynamodb:us-east-1:111122223333:table/*",
      "Condition": { "StringEquals":
        { "aws:ResourceTag/team": "${aws:PrincipalTag/team}" } } },
    { "Sid": "DenyNonTLS",
      "Effect": "Deny", "Action": "s3:*", "Resource": "*",
      "Condition": { "Bool": { "aws:SecureTransport": "false" } } } ] }
```

CODE EXPLAINED

- → Policy variables scope S3 paths to the caller's tag
- → The tag condition lets teams touch only their own tables (ABAC)
- → The explicit Deny blocks non-TLS requests even if allowed elsewhere
- → Simulate and validate before attaching to production roles

AZURE / GCP EQUIVALENT

- ↔ **Azure** Azure RBAC role definitions and assignments (az-06)
- ↔ **GCP** IAM allow / deny policies and conditions (gcp-06)
- ↔ AWS evaluates policy documents; Azure and GCP evaluate role bindings

CREATE, ATTACH, TEST

```
aws iam create-policy --policy-name app-data --policy-document file://p.json
aws iam attach-role-policy --role-name app --policy-arn <policy-arn>
aws iam simulate-principal-policy --policy-source-arn <role-arn> \
    --action-names s3:GetObject --resource-arns arn:aws:s3:::acme-uploads/x
aws accessanalyzer validate-policy --policy-type IDENTITY_POLICY \
    --policy-document file://p.json
```

COST CHECK

- $ Policies are free
- $ Overly broad `*` grants let mistakes create expensive resources
- $ SCPs can deny expensive services or Regions organization-wide

EVALUATION ORDER

STEP	RESULT
Explicit Deny anywhere	Denied
SCP / RCP does not allow	Denied
Permission boundary does not allow	Denied
Session policy does not allow	Denied
Identity or resource Allow	Allowed
Nothing matches	Denied (implicit)

POLICY TYPES

TYPE	ATTACHED TO
Identity (managed / inline)	User, group, role
Resource	S3, SQS, KMS, Lambda…
Permission boundary	User or role
SCP / RCP	OU or account
Session	AssumeRole call

DO & DON'T

- ✓ **Do** Start from AWS managed policies, then narrow with Access Analyzer policy generation
- ✓ **Do** Use conditions (tags, VPC endpoints, TLS)
- ✗ **Don't** Use `"Action": "*"` with `"Resource": "*"` for apps
- ✗ **Don't** Write inline policies you cannot reuse or audit

LOCAL TESTING

- ⌂ `aws accessanalyzer validate-policy` lints offline-authored policies
- ⌂ Policy Simulator tests before deploy
- ⌂ cfn-lint / Checkov catch broad IAM in templates

PRODUCTION CHECKLIST

- ☐ Least-privilege customer managed policies
- ☐ Permission boundaries for roles developers create
- ☐ Access Analyzer external and unused access findings reviewed

Sheet 07

Roles & Temporary Credentials

trust policies · `aws sts assume-role` · instance profiles · cross-account roles · OIDC federation (GitHub Actions) · session duration

SHORT-LIVED ACCESS

AWS STS issues temporary credentials when a principal assumes a role. EC2 gets them through an **instance profile**, ECS through a task role, Lambda through its execution role. **Cross-account roles** let one account act in another, and **OIDC federation** lets GitHub Actions, EKS pods or other identity providers assume roles with no stored secret.

KEY POINTS

- Sessions last 15 minutes to 12 hours (role setting, default 1 hour)
- Trust policy conditions narrow who can assume (repo, branch, external ID)
- Use an external ID for third parties assuming your roles
- Role chaining limits sessions to 1 hour

ASSUME A ROLE AND TRUST GITHUB

```
aws sts assume-role --role-arn arn:aws:iam::222233334444:role/audit \
    --role-session-name me --duration-seconds 3600

# One-time: register GitHub's OIDC provider in the account
aws iam create-open-id-connect-provider \
    --url https://token.actions.githubusercontent.com \
    --client-id-list sts.amazonaws.com
```

CODE EXPLAINED

- → `assume-role` returns an access key, secret and session token
- → The OIDC provider is registered once per account
- → The `sub` condition allows only the production environment of one repo
- → Profiles can chain from an SSO profile into a target role

TRUST POLICY FOR A GITHUB DEPLOY ROLE

```
{ "Version": "2012-10-17",
  "Statement": [{ "Effect": "Allow",
    "Principal": { "Federated":
      "arn:aws:iam::1111:oidc-provider/token.actions.githubusercontent.com" },
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": {
        "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
        "token.actions.githubusercontent.com:sub":
          "repo:acme/shop:environment:production" } } }] }
```

AZURE / GCP EQUIVALENT

- ↔ **Azure** managed identities + federated credentials (az-07)
- ↔ **GCP** service accounts + Workload Identity Federation (gcp-07)

COST CHECK

- $ STS and OIDC providers are free
- $ Removing static keys removes rotation work and leak risk
- $ No extra charge for instance profiles or task roles

PROFILE THAT ASSUMES A ROLE

```
# ~/.aws/config
# [profile prod-admin]
# source_profile = sso-base
# role_arn = arn:aws:iam::222233334444:role/admin
```

SECURITY CHECK

- ⚠ Pin OIDC trust to `repo:org/app:environment:production`, never `repo:org/*`
- ⚠ Require `sts:ExternalId` for vendor cross-account roles
- ⚠ Review trust policies for `"Principal": "*"`

WHERE ROLES COME FROM

COMPUTE	MECHANISM
EC2	Instance profile
ECS / Fargate	Task role
Lambda	Execution role
EKS pods	Pod Identity or IRSA
GitHub Actions	OIDC + AssumeRoleWithWebIdentity
Another account	Cross-account trust

STS CALLS

CALL	USE
AssumeRole	Principal with AWS creds
AssumeRoleWithWebIdentity	OIDC tokens
AssumeRoleWithSAML	SAML IdP
GetCallerIdentity	Who am I
GetSessionToken	MFA-protected user sessions

DO & DON'T

- ✓ **Do** Federate CI with OIDC and delete CI access keys
- ✓ **Do** Keep session durations as short as practical
- ✗ **Don't** Trust an entire GitHub org in a production role
- ✗ **Don't** Pass temporary credentials around in logs

LOCAL TESTING

- ⌂ `aws configure export-credentials --profile p --format env` prints temporary creds for tools that need env vars
- ⌂ LocalStack accepts STS calls for local tests
- ⌂ `act` can test workflows (OIDC needs the real runner)

PRODUCTION CHECKLIST

- ☐ OIDC trust conditions pinned to repo + environment
- ☐ CloudTrail alerts on AssumeRole from unexpected principals
- ☐ Cross-account roles reviewed and use external IDs for vendors

Sheet 08

Secrets, Parameters & KMS

Secrets Manager · rotation · SSM Parameter Store `SecureString` · KMS keys · envelope encryption · `aws kms encrypt`

PROTECTING SECRETS AND KEYS

Secrets Manager stores secrets with versioning and built-in **rotation** (RDS passwords, API keys via Lambda). **SSM Parameter Store** holds configuration and `SecureString` parameters, cheaper for simple needs. **KMS** manages encryption keys used by almost every AWS service; it never exposes the key material and uses **envelope encryption** for large data.

KEY POINTS

- Secrets Manager integrates with RDS, ECS, Lambda and EKS
- Parameter Store standard tier is free; advanced tier adds size and policies
- KMS key types: AWS owned, AWS managed (`aws/s3`), customer managed
- Key policies control who can use and manage each KMS key

SECURITY CHECK

- ⚠ Encrypt secrets with a customer managed KMS key where you need audit and control
- ⚠ Restrict `secretsmanager:GetSecretValue` to the workload role only
- ⚠ Enable automatic rotation for database credentials

LIMITS & QUOTAS

- ≤ Secret value up to 64 KB
- ≤ Parameter Store: 4 KB standard / 8 KB advanced per parameter
- ≤ KMS request rates are throttled per Region (cache data keys)

SECRETS, PARAMETERS, KMS

```
aws secretsmanager create-secret --name prod/shop/db \
    --kms-key-id alias/shop --secret-string file://db.json
aws secretsmanager get-secret-value --secret-id prod/shop/db \
    --query SecretString --output text
aws secretsmanager rotate-secret --secret-id prod/shop/db \
    --rotation-rules AutomaticallyAfterDays=30

aws ssm put-parameter --name /shop/prod/feature-x --type String --value on
aws ssm put-parameter --name /shop/prod/api-key --type SecureString \
    --value file://key.txt --key-id alias/shop
aws ssm get-parameters-by-path --path /shop/prod --with-decryption

aws kms create-key --description "shop data key" --key-usage ENCRYPT_DECRYPT
aws kms create-alias --alias-name alias/shop --target-key-id <key-id>
aws kms generate-data-key --key-id alias/shop --key-spec AES_256
aws kms enable-key-rotation --key-id <key-id>
```

CODE EXPLAINED

- → The secret is encrypted with the `alias/shop` customer managed key
- → Rotation uses a Lambda function (AWS provides templates for RDS)
- → Parameter paths group config by app and environment
- → `generate-data-key` returns a plaintext + encrypted data key for envelope encryption

AZURE / GCP EQUIVALENT

↔ **Azure**	Key Vault secrets, keys and references (az-08)
↔ **GCP**	Secret Manager and Cloud KMS (gcp-08)

COST CHECK

- $ Secrets Manager: per secret per month + per 10,000 API calls
- $ KMS: per customer managed key per month + requests; AWS managed keys have no monthly fee
- $ Cache secrets and data keys (AWS caching libraries) to cut calls

SECRETS MANAGER VS PARAMETER STORE

FEATURE	SECRETS MANAGER	PARAMETER STORE
Rotation	Built in	DIY
Cross-account sharing	Resource policy	Advanced tier only
Max size	64 KB	4 / 8 KB
Cost	Per secret + calls	Standard free

KMS KEY TYPES

TYPE	CONTROL
AWS owned	None (default for some services)
AWS managed (aws/service)	View only
Customer managed	Full: policy, rotation, deletion
Imported / external	You supply key material

DO & DON'T

- ✓ **Do** Fetch secrets at startup with the workload role and cache them
- ✓ **Do** Use customer managed keys for sensitive data
- ✗ **Don't** Store secrets in Lambda env vars or task definitions in plain text
- ✗ **Don't** Schedule KMS key deletion without checking what uses it

LOCAL TESTING

- ⌂ LocalStack emulates Secrets Manager, SSM and KMS
- ⌂ moto mocks them in Python tests
- ⌂ Use a gitignored `.env` locally; never real production secrets

PRODUCTION CHECKLIST

- ☐ Rotation enabled for database credentials
- ☐ KMS key rotation and deletion protection (waiting period) in place
- ☐ CloudTrail alerts on GetSecretValue from unexpected roles

Security & Audit Services

DETECT, AUDIT AND PROTECT

CloudTrail records API calls; an **organization trail** sends every account's events to one bucket. **GuardDuty** detects threats from logs and runtime signals. **AWS Config** records resource configuration and evaluates rules. **Security Hub** aggregates findings and scores against standards. **IAM Access Analyzer** finds external and unused access, and **WAF / Shield** protect web apps.

KEY POINTS

- CloudTrail event history keeps 90 days of management events for free
- Delegate admin for GuardDuty / Security Hub to a security account
- Config conformance packs bundle rules for standards
- Shield Standard is automatic; Shield Advanced is paid

TURN ON THE BASICS

```
aws cloudtrail create-trail --name org-trail --s3-bucket-name acme-ct-logs \
    --is-organization-trail --is-multi-region-trail
aws cloudtrail start-logging --name org-trail
aws cloudtrail lookup-events --lookup-attributes \
    AttributeKey=EventName,AttributeValue=DeleteBucket --max-results 5

aws guardduty create-detector --enable
aws securityhub enable-security-hub --enable-default-standards
aws configservice put-configuration-recorder --configuration-recorder \
    name=default,roleARN=arn:aws:iam::111122223333:role/config-role
aws accessanalyzer create-analyzer --analyzer-name org --type ORGANIZATION

aws securityhub get-findings --filters \
    '{"SeverityLabel":[{"Value":"CRITICAL","Comparison":"EQUALS"}]}' \
    --query "Findings[].Title" --max-items 10
```

CODE EXPLAINED

- → The organization trail covers every account and Region
- → `lookup-events` searches the last 90 days of management events
- → Security Hub default standards include AWS Foundational Security Best Practices
- → An organization analyzer reports access shared outside the org

AZURE / GCP EQUIVALENT

- ↔ **Azure** Activity Log, Defender for Cloud, Azure Policy, Sentinel (az-09)
- ↔ **GCP** Cloud Audit Logs, Security Command Center, Cloud Armor (gcp-09)

COST CHECK

- $ One CloudTrail copy of management events per Region is free to deliver; S3 storage is billed
- $ GuardDuty and Security Hub bill by volume analyzed / checks
- $ Config bills per configuration item recorded and rule evaluation

SERVICES

SERVICE	DOES
CloudTrail	API audit log
GuardDuty	Threat detection
Security Hub	Findings + standards
AWS Config	Config history + rules
Access Analyzer	External / unused access
Inspector	Vulnerability scanning
Macie	Sensitive data in S3
WAF / Shield	Web and DDoS protection

DO & DON'T

- ✓ **Do** Use an organization trail to a locked-down log archive account
- ✓ **Do** Route high-severity findings to on-call
- ✗ **Don't** Disable GuardDuty to save money in production
- ✗ **Don't** Let findings pile up without owners

LOCAL TESTING

- ⌂ Prowler and ScoutSuite audit a live account from your laptop
- ⌂ Checkov / cfn-nag scan templates before deploy
- ⌂ Test Config rules in a sandbox account

PRODUCTION CHECKLIST

- ☐ Org-wide CloudTrail, GuardDuty, Security Hub and Config
- ☐ SCP denying CloudTrail / GuardDuty disablement
- ☐ Findings triaged with SLAs

VPC Basics

CIDR blocks · public vs private subnets · route tables · Internet Gateway · default VPC · subnet per AZ

PRIVATE NETWORKING ON AWS

A **VPC** is a regional private network with one or more IPv4 CIDR blocks (and optional IPv6). Each **subnet** lives in one AZ. A subnet is public when its **route table** sends `0.0.0.0/0` to an **Internet Gateway**; private subnets reach the internet through a NAT gateway (sheet 11) or not at all. Every Region has a **default VPC** for quick tests; build your own for real work.

KEY POINTS

- AWS reserves 5 addresses in each subnet
- Use at least two AZs (three for production)
- Public IPv4 addresses are billed hourly; consider IPv6 or private designs
- VPC Flow Logs capture accepted / rejected traffic

SECURITY CHECK

- ⚠ Keep databases and app servers in private subnets
- ⚠ Disable auto-assign public IP on private subnets
- ⚠ Enable VPC Flow Logs

LIMITS & QUOTAS

- ≤ 5 VPCs per Region by default (adjustable)
- ≤ 200 subnets per VPC
- ≤ 5 IPv4 CIDR blocks per VPC by default (up to 50)

VPC, SUBNETS, INTERNET ROUTE

```
VPC=$(aws ec2 create-vpc --cidr-block 10.30.0.0/16 \
  --tag-specifications 'ResourceType=vpc,Tags=[{Key=Name,Value=shop-prod}]' \
  --query Vpc.VpcId --output text)
PUB_A=$(aws ec2 create-subnet --vpc-id $VPC --cidr-block 10.30.0.0/24 \
  --availability-zone us-east-1a --query Subnet.SubnetId --output text)
APP_A=$(aws ec2 create-subnet --vpc-id $VPC --cidr-block 10.30.10.0/24 \
  --availability-zone us-east-1a --query Subnet.SubnetId --output text)

IGW=$(aws ec2 create-internet-gateway \
  --query InternetGateway.InternetGatewayId \
  --output text)
aws ec2 attach-internet-gateway --vpc-id $VPC --internet-gateway-id $IGW
RT=$(aws ec2 create-route-table --vpc-id $VPC --query RouteTable.RouteTableId \
  --output text)
aws ec2 create-route --route-table-id $RT --destination-cidr-block 0.0.0.0/0 \
    --gateway-id $IGW
aws ec2 associate-route-table --route-table-id $RT --subnet-id $PUB_A

aws ec2 create-flow-logs --resource-type VPC --resource-ids $VPC \
    --traffic-type ALL --log-destination-type cloud-watch-logs \
    --log-group-name /vpc/shop-prod --deliver-logs-permission-arn <role-arn>
```

CODE EXPLAINED

- → Each subnet is pinned to one AZ
- → Only the public subnet's route table points at the Internet Gateway
- → Private subnets keep the main route table (local only) until NAT is added
- → Flow logs go to CloudWatch Logs through an IAM role

AZURE / GCP EQUIVALENT

- ↔ **Azure** VNet with zone-spanning subnets (az-10)
- ↔ **GCP** global VPC with regional subnets (gcp-10)
- ↔ AWS subnets are per-AZ; Azure and GCP subnets span zones

COST CHECK

- $ VPCs, subnets and route tables are free
- $ Public IPv4 addresses bill hourly, even on running instances
- $ Cross-AZ traffic bills per GB in each direction

PIECES

PIECE	ROLE
VPC	Regional network + CIDR
Subnet	One AZ, a slice of the CIDR
Route table	Where traffic goes
Internet Gateway	Public internet path
NAT Gateway	Private outbound (sheet 11)
VPC endpoint	Private AWS service access (sheet 11)

TYPICAL LAYOUT

TIER	SUBNETS
Public	Load balancers, NAT
App (private)	EC2 / ECS / Lambda ENIs
Data (private)	RDS, ElastiCache
Per AZ	Repeat each tier in 2–3 AZs

DO & DON'T

- ✓ **Do** Plan CIDRs that never overlap with other VPCs or on-premises
- ✓ **Do** Build VPCs with IaC (VPC module / CDK construct)
- ✗ **Don't** Run production in the default VPC
- ✗ **Don't** Make every subnet public "to keep it simple"

LOCAL TESTING

- ⌂ Reachability Analyzer explains why traffic can or cannot flow
- ⌂ Docker networks mimic tiers locally
- ⌂ LocalStack supports EC2 / VPC API calls for IaC tests

PRODUCTION CHECKLIST

- ☐ Three AZs, public / private / data tiers
- ☐ Flow logs enabled and retained
- ☐ No public IPs on private tiers

Sheet 11

NAT, Endpoints & Connectivity

NAT gateway · gateway vs interface VPC endpoints (PrivateLink) · VPC peering · Transit Gateway · Site-to-Site VPN

GETTING OUT, REACHING AWS PRIVATELY, CONNECTING NETWORKS

A **NAT gateway** in a public subnet gives private subnets outbound internet access. **VPC endpoints** reach AWS services without the internet: **gateway endpoints** for S3 and DynamoDB (free, via route tables) and **interface endpoints** (PrivateLink ENIs) for most other services. **Peering** links two VPCs; **Transit Gateway** is a hub for many VPCs and VPNs.

KEY POINTS

- One NAT gateway per AZ avoids cross-AZ dependency and charges
- Gateway endpoints cut NAT data charges for S3 / DynamoDB traffic
- Interface endpoints need private DNS enabled to keep normal service names
- Peering is not transitive; Transit Gateway routes between many

NAT, ENDPOINTS, PEERING

```
EIP=$(aws ec2 allocate-address --query AllocationId --output text)
NAT=$(aws ec2 create-nat-gateway --subnet-id $PUB_A --allocation-id $EIP \
  --query NatGateway.NatGatewayId --output text)
aws ec2 create-route --route-table-id $APP_RT \
    --destination-cidr-block 0.0.0.0/0 --nat-gateway-id $NAT

# Free gateway endpoint for S3
aws ec2 create-vpc-endpoint --vpc-id $VPC --vpc-endpoint-type Gateway \
    --service-name com.amazonaws.us-east-1.s3 --route-table-ids $APP_RT

# Interface endpoint (PrivateLink) for Secrets Manager
aws ec2 create-vpc-endpoint --vpc-id $VPC --vpc-endpoint-type Interface \
    --service-name com.amazonaws.us-east-1.secretsmanager \
    --subnet-ids $APP_A $APP_B --security-group-ids $SG_ENDPOINTS \
    --private-dns-enabled

# Peer two VPCs
aws ec2 create-vpc-peering-connection --vpc-id $VPC --peer-vpc-id $VPC2
aws ec2 accept-vpc-peering-connection --vpc-peering-connection-id pcx-0abc
```

CODE EXPLAINED

- → The NAT gateway needs an Elastic IP and lives in a public subnet
- → Private route table sends internet traffic to the NAT gateway
- → The S3 gateway endpoint is a route-table entry, no ENIs
- → Interface endpoints create ENIs in your subnets with private DNS

AZURE / GCP EQUIVALENT

- ↔ **Azure** NAT Gateway, Private Endpoints, VNet peering, Virtual WAN (az-11)
- ↔ **GCP** Cloud NAT, Private Service Connect, peering, NCC (gcp-11)

COST CHECK

- $ NAT gateway: hourly + per GB processed, even when idle
- $ Interface endpoints: hourly per AZ + per GB; gateway endpoints are free
- $ Transit Gateway: per attachment-hour + per GB

CONNECTIVITY OPTIONS

NEED	USE
Private subnet → internet	NAT gateway
Private → S3 / DynamoDB	Gateway endpoint
Private → other AWS APIs	Interface endpoint
VPC ↔ VPC (few)	Peering
Many VPCs + on-prem	Transit Gateway
Office ↔ AWS	Site-to-Site VPN / Direct Connect

DO & DON'T

- ✓ **Do** Add S3 and DynamoDB gateway endpoints to every VPC
- ✓ **Do** Put a NAT gateway in each AZ for production
- ✗ **Don't** Leave NAT gateways in idle dev VPCs (they bill hourly)
- ✗ **Don't** Forget security groups on interface endpoints

LOCAL TESTING

- ⌂ Reachability Analyzer and Network Access Analyzer test paths
- ⌂ Price NAT vs endpoints in the Pricing Calculator
- ⌂ IaC plans show endpoint and route changes before apply

PRODUCTION CHECKLIST

- ☐ NAT per AZ; gateway endpoints for S3 / DynamoDB
- ☐ Interface endpoints for sensitive service APIs
- ☐ Routing documented; non-overlapping CIDRs

Security Groups & NACLs

FILTERING TRAFFIC

Security groups are stateful firewalls attached to ENIs (instances, load balancers, RDS, Lambda in a VPC). They only allow; replies are automatic. Rules can reference other security groups, so "the app tier may reach the database" needs no IPs. **Network ACLs** are stateless allow / deny lists on subnets, evaluated in rule-number order, and must allow return traffic on ephemeral ports.

KEY POINTS

- New security groups deny all inbound and allow all outbound
- SG references follow instances as they scale
- NACLs are a coarse second layer; most designs leave the default allow-all NACL
- Flow Logs show ACCEPT / REJECT decisions

SECURITY CHECK

- ⚠ Never allow 22 / 3389 from 0.0.0.0/0; use Session Manager
- ⚠ Restrict outbound on data-tier security groups
- ⚠ Alert on security group changes that open ports to the world

LIMITS & QUOTAS

- ≤ 60 inbound and 60 outbound rules per security group (adjustable)
- ≤ 5 security groups per network interface (up to 16)
- ≤ 20 rules per NACL direction by default

TIERED SECURITY GROUPS

```
ALB_SG=$(aws ec2 create-security-group --vpc-id $VPC --group-name alb \
  --description "public ALB" --query GroupId --output text)
APP_SG=$(aws ec2 create-security-group --vpc-id $VPC --group-name app \
  --description "app tier" --query GroupId --output text)
DB_SG=$(aws ec2 create-security-group --vpc-id $VPC --group-name db \
  --description "database" --query GroupId --output text)

aws ec2 authorize-security-group-ingress --group-id $ALB_SG \
    --protocol tcp --port 443 --cidr 0.0.0.0/0
aws ec2 authorize-security-group-ingress --group-id $APP_SG \
    --protocol tcp --port 8080 --source-group $ALB_SG
aws ec2 authorize-security-group-ingress --group-id $DB_SG \
    --protocol tcp --port 5432 --source-group $APP_SG
aws ec2 revoke-security-group-egress --group-id $DB_SG \
    --ip-permissions '[{"IpProtocol":"-1","IpRanges":[{"CidrIp":"0.0.0.0/0"}]}]'

aws ec2 describe-security-group-rules \
    --filters Name=group-id,Values=$APP_SG --output table
```

CODE EXPLAINED

- → Only the ALB is open to the internet
- → The app tier accepts traffic only from the ALB's group
- → The database accepts only the app tier on 5432
- → Removing default egress stops the database from calling out

AZURE / GCP EQUIVALENT

- ↔ **Azure** NSGs with service tags and ASGs (az-12)
- ↔ **GCP** VPC firewall rules / policies with tags and service accounts (gcp-12)

COST CHECK

- $ Security groups and NACLs are free
- $ Flow Logs bill for log ingestion / storage
- $ AWS Network Firewall (central inspection) bills hourly + per GB

SG VS NACL

FEATURE	SECURITY GROUP	NACL
Attached to	ENI	Subnet
State	Stateful	Stateless
Rules	Allow only	Allow and deny
Order	All evaluated	Lowest number first
References	Other SGs	CIDRs only

COMMON PORTS

PORT	SERVICE
443	HTTPS
80	HTTP (redirect)
5432 / 3306	PostgreSQL / MySQL
6379	Redis / Valkey
1024-65535	Ephemeral (NACL return)

DO & DON'T

- ✓ **Do** Reference security groups instead of IP ranges inside the VPC
- ✓ **Do** Name and describe every group clearly
- ✗ **Don't** Open 0.0.0.0/0 on anything but public load balancers
- ✗ **Don't** Add deny rules to NACLs without allowing ephemeral return ports

LOCAL TESTING

- ⌂ Reachability Analyzer shows which rule blocks a path
- ⌂ Flow Logs + Logs Insights find rejected connections
- ⌂ `nc -zv host port` from inside the VPC tests rules

PRODUCTION CHECKLIST

- ☐ No admin ports open to the internet
- ☐ Data-tier egress restricted
- ☐ Config rule / Security Hub control for open security groups

Route 53

hosted zones · A / AAAA / CNAME / MX / TXT records · alias records · routing policies (weighted, latency, failover) · health checks · domain registration

DNS ON AWS

Route 53 hosts public and private **hosted zones**, registers domains, and routes with policies: **simple**, **weighted**, **latency**, **failover**, **geolocation** and more, often combined with **health checks**. **Alias records** point names (even the zone apex) at AWS resources like CloudFront, load balancers and S3 websites, and follow their IP changes for free.

KEY POINTS

- Private hosted zones resolve only inside associated VPCs
- Alias queries to AWS resources are free
- Route 53 Resolver handles hybrid DNS (inbound / outbound endpoints)
- Route 53 has a 100% availability SLA

ZONE, ALIAS RECORD, FAILOVER

```
aws route53 create-hosted-zone --name acme-shop.com \
    --caller-reference $(date +%s)
aws route53 list-hosted-zones --query "HostedZones[].[Name,Id]" --output table
```

CHANGE BATCH: ALIAS TO AN ALB (ALIAS.JSON)

```
{ "Changes": [{ "Action": "UPSERT",
  "ResourceRecordSet": { "Name": "shop.acme-shop.com", "Type": "A",
    "SetIdentifier": "primary", "Failover": "PRIMARY",
    "HealthCheckId": "11111111-2222-3333-4444-555555555555",
    "AliasTarget": { "HostedZoneId": "Z35SXDOTRQ7X7K",
      "DNSName": "web-alb-123.us-east-1.elb.amazonaws.com",
      "EvaluateTargetHealth": true } } }] }
```

APPLY AND CHECK

```
aws route53 create-health-check --caller-reference hc-$(date +%s) \
    --health-check-config file://health-check.json
aws route53 change-resource-record-sets --hosted-zone-id Z0123 \
    --change-batch file://alias.json
dig +short shop.acme-shop.com
```

CODE EXPLAINED

- → `caller-reference` must be unique per request
- → The alias target uses the load balancer's own hosted zone ID
- → A matching SECONDARY record completes failover routing
- → Health checks decide when to fail over

AZURE / GCP EQUIVALENT

- ↔ **Azure** Azure DNS + Traffic Manager (az-13)
- ↔ **GCP** Cloud DNS + routing policies (gcp-13)

COST CHECK

- $ Hosted zone per month + per million queries
- $ Alias queries to AWS resources are free
- $ Health checks bill per check per month

ROUTING POLICIES

POLICY	ROUTES BY
Simple	One answer
Weighted	Percentages
Latency	Lowest latency Region
Failover	Primary / secondary with health
Geolocation	User location
Geoproximity	Distance + bias
Multivalue	Several healthy IPs
IP-based	Client CIDR

RECORD TYPES

TYPE	USE
A / AAAA	IPv4 / IPv6
CNAME	Alias name (not apex)
Alias	AWS resource, apex OK
MX / TXT	Mail, verification
CAA	Allowed CAs

DO & DON'T

- ✓ **Do** Use alias records for AWS targets
- ✓ **Do** Keep TTLs short on failover records
- ✗ **Don't** Point a CNAME at the zone apex
- ✗ **Don't** Delete a hosted zone still delegated at the registrar

LOCAL TESTING

- ⌂ `dig @ns-123.awsdns-45.com shop.acme-shop.com` tests before delegation
- ⌂ Route 53 "test record" in the console simulates answers
- ⌂ CoreDNS locally for split-horizon experiments

PRODUCTION CHECKLIST

- ☐ Records managed in IaC
- ☐ Health-checked failover for critical names
- ☐ DNSSEC signing considered; registrar lock on

CloudFront & ACM

CDN AND CERTIFICATES

CloudFront caches content at hundreds of edge locations in front of **origins** (S3, ALB, API Gateway, any HTTP server). **Behaviors** map path patterns to origins and cache / origin-request policies. **Origin Access Control (OAC)** lets only CloudFront read a private S3 bucket. **ACM** issues free, auto-renewing TLS certificates; CloudFront certificates must be in **us-east-1**.

KEY POINTS

- Managed cache policies (CachingOptimized, CachingDisabled) cover common cases
- CloudFront Functions run lightweight JS at the edge; Lambda@Edge for heavier logic
- Invalidations are free for the first 1,000 paths per month
- Price classes limit edge locations to cut cost

SECURITY CHECK

- ⚠ Keep the S3 origin private with OAC and Block Public Access
- ⚠ Redirect HTTP to HTTPS and use a modern TLS security policy
- ⚠ Attach AWS WAF to public distributions

CERTIFICATE, DISTRIBUTION, INVALIDATION

```
aws acm request-certificate --region us-east-1 \
    --domain-name shop.acme-shop.com --validation-method DNS
aws acm describe-certificate --region us-east-1 --certificate-arn <arn> \
    --query "Certificate.DomainValidationOptions[].ResourceRecord"

aws cloudfront create-origin-access-control \
    --origin-access-control-config file://oac.json   # sigv4, always, s3
aws cloudfront create-distribution --distribution-config file://dist.json

aws s3 sync ./dist s3://acme-site --delete \
    --cache-control "public,max-age=31536000,immutable" --exclude index.html
aws s3 cp ./dist/index.html s3://acme-site/ --cache-control "no-cache"
aws cloudfront create-invalidation --distribution-id E1ABCDEF \
    --paths /index.html
```

CODE EXPLAINED

- → The DNS validation record proves you own the domain; ACM renews automatically
- → OAC signs CloudFront's requests to S3
- → Hashed assets get a one-year cache; `index.html` revalidates
- → Only `index.html` needs invalidating after a deploy

AZURE / GCP EQUIVALENT

- ↔ **Azure** Front Door Standard / Premium + managed certificates (az-14)
- ↔ **GCP** Cloud CDN + Certificate Manager on a global ALB (gcp-14)

COST CHECK

- $ Data transfer out to the internet + requests; a monthly free allowance applies
- $ ACM public certificates are free
- $ Flat-rate plans bundle CloudFront, WAF and more for predictable bills

DISTRIBUTION PIECES

PIECE	ROLE
`Origin`	S3, ALB, API, custom
`Behavior`	Path → origin + policies
`Cache policy`	TTLs and cache key
`Origin request policy`	Headers / cookies forwarded
`Response headers policy`	Security headers, CORS
`OAC`	Private S3 access

EDGE COMPUTE

OPTION	USE
`CloudFront Functions`	Rewrites, redirects, headers (sub-ms)
`Lambda@Edge`	Auth, origin selection, larger logic
`KeyValueStore`	Config for CloudFront Functions

DO & DON'T

- ✓ **Do** Use versioned asset names and long TTLs
- ✓ **Do** Request ACM certificates in us-east-1 for CloudFront
- ✗ **Don't** Leave the S3 bucket public behind CloudFront
- ✗ **Don't** Invalidate `/*` on every deploy

LOCAL TESTING

- ⌂ `npx serve dist` previews the site
- ⌂ `curl -I` shows `x-cache: Hit from cloudfront` once deployed
- ⌂ Test CloudFront Functions in the console test tool

PRODUCTION CHECKLIST

- ☐ OAC + Block Public Access on the origin bucket
- ☐ WAF attached; HTTPS-only with TLSv1.2_2021 or newer
- ☐ Standard logs or real-time logs enabled

EC2 Basics

VIRTUAL SERVERS ON AWS

An EC2 instance combines an **instance type** (family, generation, size), an **AMI** (machine image), EBS volumes, a subnet and security groups. Type names encode the details: `m7g.large` is general purpose (m), generation 7, Graviton Arm (g), large. **User data** runs a script at first boot. **IMDSv2** protects the metadata service that hands out role credentials.

KEY POINTS

- Graviton (Arm) types usually give the best price-performance for Linux
- Get current Amazon Linux AMIs from public SSM parameters
- Use Session Manager instead of SSH keys where possible
- Burstable T types earn and spend CPU credits

SECURITY CHECK

- ⚠ Require IMDSv2 (`HttpTokens=required`) on every instance
- ⚠ No public IP and no port 22: use Session Manager
- ⚠ Encrypt EBS volumes by default (account setting)

LIMITS & QUOTAS

- ≤ On-Demand vCPU quotas per instance family group and Region
- ≤ New accounts start low: check `aws service-quotas list-service-quotas --service-code ec2`
- ≤ 5 Elastic IPs per Region by default

LAUNCH A PRIVATE GRAVITON INSTANCE

```
P=/aws/service/ami-amazon-linux-latest
AMI=resolve:ssm:$P/al2023-ami-kernel-default-arm64
aws ec2 run-instances --image-id $AMI --instance-type t4g.small \
    --subnet-id $APP_A --security-group-ids $APP_SG \
    --iam-instance-profile Name=app-instance-profile \
    --metadata-options HttpTokens=required,HttpEndpoint=enabled \
    --user-data file://init.sh --no-associate-public-ip-address \
    --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=web-1}]'

aws ec2 describe-instances --filters Name=tag:Name,Values=web-1 \
    --query "Reservations[].Instances[].[InstanceId,State.Name]"
aws ssm start-session --target i-0abc1234                # shell, no SSH
aws ec2 enable-ebs-encryption-by-default
aws service-quotas get-service-quota --service-code ec2 \
    --quota-code L-1216C47A    # Running On-Demand Standard instances (vCPUs)
```

CODE EXPLAINED

- → The `resolve:ssm:` prefix looks up the latest Amazon Linux 2023 Arm AMI
- → The instance profile gives the instance its IAM role (and Session Manager access)
- → IMDSv2 required blocks token-less metadata requests
- → L-1216C47A is the vCPU quota for standard families

AZURE / GCP EQUIVALENT

- ↔ **Azure** Virtual Machines, cloud-init, Bastion (az-15)
- ↔ **GCP** Compute Engine, startup scripts, OS Login / IAP (gcp-15)

COST CHECK

- $ Billed per second (Linux) while running, plus EBS and public IPv4
- $ Graviton types cost less per hour than comparable x86
- $ Stopped instances still pay for EBS volumes and Elastic IPs

INIT.SH (USER DATA)

```
#!/bin/bash
dnf install -y nginx && systemctl enable --now nginx
```

TYPE NAME DECODER

PART	MEANING
`m / c / r / t / g / p`	General / compute / memory / burstable / GPU
`7`	Generation
`g`	Graviton (Arm)
`i / a`	Intel / AMD
`d`	Local NVMe
`n`	Enhanced networking
`.large`	Size

ACCESS OPTIONS

OPTION	USE
`Session Manager`	Default; audited, no ports
`EC2 Instance Connect`	Short-lived SSH keys
`SSH key pair`	Legacy
`Serial console`	Boot problems

DO & DON'T

- ✓ **Do** Bake AMIs with EC2 Image Builder or Packer
- ✓ **Do** Use launch templates for every instance
- ✗ **Don't** Assign public IPs to app servers
- ✗ **Don't** Hand-patch long-lived pets

LOCAL TESTING

- ⌂ Test user data in a local VM or container first
- ⌂ LocalStack EC2 emulation for IaC flows (no real VMs)
- ⌂ Packer builds and tests AMIs from your laptop

PRODUCTION CHECKLIST

- ☐ IMDSv2, EBS encryption, no public IPs
- ☐ SSM Agent + Patch Manager baselines
- ☐ Instances in Auto Scaling groups across AZs

EC2 Operations & Pricing

Session Manager · EBS volumes & snapshots · stop vs terminate · Elastic IPs · On-Demand / Spot / Savings Plans · Reserved Instances

RUNNING AND PAYING FOR EC2

Instances can be **stopped** (EBS kept, compute not billed) or **terminated** (gone; root volume deleted by default). **EBS** volumes and **snapshots** hold data. Pricing options: **On-Demand**; **Savings Plans** (1 or 3 year spend commitment, flexible); **Reserved Instances** (specific type); and **Spot** (spare capacity at a deep discount with a 2-minute interruption notice).

KEY POINTS

- Snapshots are incremental and stored in S3 (managed by AWS)
- Termination protection guards important instances
- Spot works best for stateless, fault-tolerant work
- Compute Savings Plans also cover Fargate and Lambda

OPERATE AND SAVE

```
aws ec2 stop-instances --instance-ids i-0abc1234       # compute billing stops
aws ec2 start-instances --instance-ids i-0abc1234
aws ec2 modify-instance-attribute --instance-id i-0abc1234 \
    --disable-api-termination

aws ec2 create-volume --availability-zone us-east-1a --size 100 \
    --volume-type gp3 --iops 6000 --throughput 250 --encrypted
aws ec2 attach-volume --volume-id vol-0abc --instance-id i-0abc1234 \
    --device /dev/sdf
aws ec2 create-snapshot --volume-id vol-0abc --description nightly
aws dlm create-lifecycle-policy --execution-role-arn <role-arn> \
    --description daily --state ENABLED --policy-details file://dlm.json

# Spot instance
aws ec2 run-instances --image-id $AMI --instance-type c7g.large \
    --instance-market-options MarketType=spot

aws ec2 describe-volumes --filters Name=status,Values=available \
    --query "Volumes[].[VolumeId,Size]" --output table     # unattached
aws ce get-savings-plans-purchase-recommendation \
    --savings-plans-type COMPUTE_SP --term-in-years ONE_YEAR \
    --payment-option NO_UPFRONT --lookback-period-in-days THIRTY_DAYS
```

CODE EXPLAINED

- → Stopping keeps EBS volumes; terminating usually deletes the root volume
- → gp3 sets IOPS and throughput independently of size
- → Data Lifecycle Manager automates snapshot schedules
- → Cost Explorer suggests a Savings Plan from recent usage

AZURE / GCP EQUIVALENT

- ↔ **Azure** deallocate, managed disks, Spot VMs, Reservations / savings plan (az-16)
- ↔ **GCP** Persistent Disk / Hyperdisk, Spot VMs, committed use discounts (gcp-16)

COST CHECK

- $ Savings Plans: up to ~72% off vs On-Demand
- $ Spot: up to ~90% off, interruptible
- $ Unattached EBS volumes and idle Elastic IPs keep billing

PRICING OPTIONS

OPTION	COMMIT	BEST FOR
On-Demand	None	Spiky, short-lived
Compute Savings Plan	$/hour, 1–3 yr	Steady, flexible types
EC2 Instance Savings Plan	Family + Region	Steady, fixed family
Reserved Instances	Type, 1–3 yr	Legacy commitments
Spot	None	Batch, CI, stateless

EBS TYPES

TYPE	USE
gp3	Default SSD
io2 Block Express	High IOPS databases
st1	Throughput HDD
sc1	Cold HDD

DO & DON'T

- ✓ **Do** Use gp3 instead of gp2
- ✓ **Do** Automate snapshots with DLM or AWS Backup
- ✗ **Don't** Leave detached volumes and old snapshots around
- ✗ **Don't** Buy commitments before rightsizing with Compute Optimizer

LOCAL TESTING

- ⌂ Price options in the AWS Pricing Calculator
- ⌂ Test Spot interruption handling with Fault Injection Service
- ⌂ Restore a snapshot into a scratch instance to verify backups

PRODUCTION CHECKLIST

- ☐ Backups / snapshot policies on all data volumes
- ☐ Savings Plans cover the steady baseline
- ☐ Alerts on idle and unattached resources

Elastic Load Balancing

SPREADING TRAFFIC ACROSS TARGETS

Application Load Balancers (ALB) work at layer 7: HTTP / HTTPS routing by host, path, headers or query, with TLS termination, authentication and WAF. **Network Load Balancers (NLB)** work at layer 4 with static IPs, very high throughput and TLS passthrough. Both send traffic to **target groups** (instances, IPs, Lambda) checked by **health checks**.

KEY POINTS

- Listeners accept connections on a port; rules route to target groups
- ALB can authenticate users with Cognito or any OIDC provider
- Deregistration delay drains connections before removing targets
- Gateway Load Balancer inserts third-party appliances

ALB WITH HTTPS AND PATH ROUTING

```
ALB=$(aws elbv2 create-load-balancer --name shop-alb --type application \
  --subnets $PUB_A $PUB_B --security-groups $ALB_SG \
  --query "LoadBalancers[0].LoadBalancerArn" --output text)
TG=$(aws elbv2 create-target-group --name shop-api --protocol HTTP \
  --port 8080 --vpc-id $VPC --target-type ip \
  --health-check-path /health --health-check-interval-seconds 15 \
  --query "TargetGroups[0].TargetGroupArn" --output text)

aws elbv2 create-listener --load-balancer-arn $ALB --protocol HTTPS \
    --port 443 --certificates CertificateArn=$CERT \
    --ssl-policy ELBSecurityPolicy-TLS13-1-2-2021-06 \
    --default-actions Type=forward,TargetGroupArn=$TG
aws elbv2 create-listener --load-balancer-arn $ALB --protocol HTTP --port 80 \
    --default-actions file://redirect-https.json        # 301 → HTTPS

aws elbv2 create-rule --listener-arn $HTTPS_LISTENER --priority 10 \
    --conditions Field=path-pattern,Values='/admin/*' \
    --actions Type=forward,TargetGroupArn=$ADMIN_TG
aws elbv2 describe-target-health --target-group-arn $TG
```

CODE EXPLAINED

- → `target-type ip` suits Fargate and containers
- → The HTTPS listener uses an ACM certificate and a TLS 1.3 policy
- → Port 80 redirects to HTTPS
- → A path rule sends `/admin/*` to a separate target group

AZURE / GCP EQUIVALENT

↔	**Azure**	Azure Load Balancer, Application Gateway (az-17)
↔	**GCP**	Network LB, Application LB (gcp-17)

COST CHECK

- $ Hourly fee plus LCUs (ALB) / NLCUs (NLB) for traffic
- $ One ALB can serve many apps via host / path rules
- $ Cross-zone traffic for NLB can add data charges

ALB VS NLB

FEATURE	ALB	NLB
Layer	7 (HTTP)	4 (TCP / UDP / TLS)
Routing	Host, path, header	Port
Static IP	No (use Global Accelerator)	Yes
WAF / auth	Yes	No
Targets	Instance, IP, Lambda	Instance, IP, ALB

HEALTH CHECK SETTINGS

SETTING	TYPICAL
Path	/health
Interval	15–30 s
Healthy threshold	2–5
Unhealthy threshold	2
Matcher	200-399

DO & DON'T

- ✓ **Do** Redirect HTTP to HTTPS and use modern TLS policies
- ✓ **Do** Spread across at least two AZs
- ✗ **Don't** Point health checks at a page that hits the database heavily
- ✗ **Don't** Create one ALB per microservice when rules can share one

LOCAL TESTING

- ⌂ Traefik or nginx in Docker Compose mimics host / path routing
- ⌂ LocalStack supports ELBv2 APIs for IaC tests
- ⌂ `curl -H "Host: …"` tests host rules before DNS

PRODUCTION CHECKLIST

- ☐ HTTPS only, WAF attached, access logs to S3
- ☐ Deletion protection on
- ☐ Alarms on 5xx and unhealthy host count

Auto Scaling

launch templates · Auto Scaling groups · desired / min / max · target tracking · scheduled scaling · instance refresh

GROUPS THAT HEAL AND SCALE

An **Auto Scaling group (ASG)** keeps a number of instances running from a **launch template**, across AZs, replacing unhealthy ones. **Target tracking** policies keep a metric (CPU, requests per target) near a value; **scheduled** actions scale for known peaks; **predictive** scaling forecasts. **Instance refresh** rolls out a new template gradually.

KEY POINTS

- Desired capacity floats between min and max
- Use ELB health checks so failed apps (not just failed VMs) are replaced
- Mixed instances policies combine On-Demand and Spot across types
- Warm pools cut scale-out time

LIMITS & QUOTAS

- ≤ 500 Auto Scaling groups per Region by default
- ≤ 50 scaling policies per group
- ≤ 5,000 launch templates per Region

LAUNCH TEMPLATE, ASG, POLICIES

```
aws ec2 create-launch-template --launch-template-name web \
    --launch-template-data file://lt.json
aws autoscaling create-auto-scaling-group --auto-scaling-group-name web \
    --launch-template LaunchTemplateName=web,Version='$Latest' \
    --min-size 2 --max-size 10 --desired-capacity 2 \
    --vpc-zone-identifier "$APP_A,$APP_B,$APP_C" \
    --target-group-arns $TG --health-check-type ELB \
    --health-check-grace-period 120

aws autoscaling put-scaling-policy --auto-scaling-group-name web \
    --policy-name cpu50 --policy-type TargetTrackingScaling \
    --target-tracking-configuration '{"PredefinedMetricSpecification":
      {"PredefinedMetricType":"ASGAverageCPUUtilization"},"TargetValue":50}'
aws autoscaling put-scheduled-update-group-action \
    --auto-scaling-group-name web --scheduled-action-name morning \
    --recurrence "0 12 * * MON-FRI" --min-size 4

aws autoscaling start-instance-refresh --auto-scaling-group-name web \
    --preferences '{"MinHealthyPercentage":90,"InstanceWarmup":120}'
```

CODE EXPLAINED

- → The ASG spans three private subnets (three AZs)
- → ELB health checks replace instances whose app fails
- → Target tracking adds or removes instances to keep CPU near 50%
- → Instance refresh replaces instances in batches with the new template

AZURE / GCP EQUIVALENT

- ↔ **Azure** VM Scale Sets + autoscale rules (az-18)
- ↔ **GCP** Managed instance groups + autoscaler (gcp-18)

COST CHECK

- $ Auto Scaling itself is free; you pay for instances
- $ Mixed On-Demand + Spot cuts cost for stateless fleets
- $ Scheduled scale-in overnight saves on predictable load

SCALING POLICIES

POLICY	USE
`Target tracking`	Keep a metric at a target
`Step scaling`	Bigger jumps for bigger breaches
`Scheduled`	Known daily / weekly peaks
`Predictive`	Forecast from history
`Manual`	Set desired capacity

CAPACITY SETTINGS

SETTING	MEANING
`min`	Never fewer
`max`	Never more (cost cap)
`desired`	Current target
`warm pool`	Pre-initialized instances

DO & DON'T

- ✓ **Do** Use ELB health checks and a realistic grace period
- ✓ **Do** Scale on requests per target for web tiers
- ✗ **Don't** Set min = max and lose self-healing benefits of scaling
- ✗ **Don't** Store state on instances that may be terminated

LOCAL TESTING

- ⌂ Load-test with k6 / Locust against a staging ASG
- ⌂ Test launch template user data locally first
- ⌂ Review scaling activity with `aws autoscaling describe-scaling-activities`

PRODUCTION CHECKLIST

- ☐ Min 2+ across 3 AZs, ELB health checks
- ☐ Instance refresh for deployments
- ☐ Alarms on failed launches and max capacity reached

Choosing Compute

PICKING THE RIGHT AWS HOST

AWS has a host at every level of control. **Lambda** runs functions on events. **ECS on Fargate** runs containers without servers; **App Runner** deploys a container or repo as a web service with almost no configuration. **EKS** gives Kubernetes. **EC2** gives full control. **Elastic Beanstalk** and **Lightsail** are simpler packaged options.

KEY POINTS

- Fargate removes node management for ECS and EKS
- Lambda scales to zero; ECS / EKS services usually do not
- App Runner suits small HTTP services with low ops needs
- AWS Batch runs batch jobs on EC2 or Fargate

ONE CONTAINER, SEVERAL HOSTS

```
IMG=111122223333.dkr.ecr.us-east-1.amazonaws.com/shop-api:1.4.0

# App Runner: container to HTTPS URL (config in a JSON file)
aws apprunner create-service --service-name shop-api \
    --source-configuration file://apprunner.json
# apprunner.json: {"ImageRepository": {"ImageIdentifier": "<IMG>",
#   "ImageRepositoryType": "ECR", "ImageConfiguration": {"Port": "8080"}},
#   "AuthenticationConfiguration": {"AccessRoleArn": "<ecr-access-role>"}}

# ECS on Fargate (sheet 26)
aws ecs create-service --cluster shop --service-name api \
    --task-definition shop-api:7 --desired-count 2 --launch-type FARGATE \
    --network-configuration "awsvpcConfiguration={subnets=[$APP_A],\
securityGroups=[$APP_SG]}"

# Lambda from a container image (sheet 21)
aws lambda create-function --function-name shop-api --package-type Image \
    --code ImageUri=$IMG --role $LAMBDA_ROLE
```

CODE EXPLAINED

- → App Runner needs a role that can pull from ECR
- → ECS services keep a desired number of Fargate tasks running
- → Lambda can run container images up to 10 GB
- → The same image can target several services

AZURE / GCP EQUIVALENT

- ↔ **Azure** Choosing Compute: VMs vs App Service vs Functions vs Container Apps vs AKS (az-19)
- ↔ **GCP** Choosing Compute: Compute Engine vs Cloud Run vs GKE (gcp-19)

COST CHECK

- $ Lambda: per request + GB-second, free tier each month
- $ Fargate: per vCPU-hour and GB-hour while tasks run
- $ EC2 / EKS nodes bill while running; EKS adds a cluster-hour fee

DECISION TABLE

WORKLOAD	BEST FIT
Events, glue, APIs with bursty traffic	Lambda
Containerized web service, low ops	App Runner / ECS on Fargate
Many services, AWS-native	ECS
Kubernetes ecosystem	EKS
OS control, GPUs, licensing	EC2
Simple VPS / small site	Lightsail
Batch	AWS Batch

OPS BURDEN

SERVICE	YOU MANAGE
Lambda	Code
App Runner	Container
ECS on Fargate	Task definitions
EKS	Cluster add-ons, nodes (unless Auto Mode)
EC2	Everything above the hypervisor

DO & DON'T

- ✓ **Do** Default to Lambda or Fargate for new services
- ✓ **Do** Containerize to keep options open
- ✗ **Don't** Choose EKS for a single small service
- ✗ **Don't** Run long CPU-heavy jobs on Lambda

LOCAL TESTING

- ⌂ SAM CLI for Lambda, Docker Compose for containers, kind for Kubernetes
- ⌂ AWS Copilot CLI scaffolds ECS services
- ⌂ Compare costs in the Pricing Calculator

PRODUCTION CHECKLIST

- ☐ Host choice documented with expected cost and limits
- ☐ Autoscaling and health checks on the chosen service
- ☐ Same image promoted across environments

Lambda Basics

handler signature · runtimes · deploy a zip · `aws lambda invoke` · environment variables · memory & timeout · function URLs

FUNCTIONS ON AWS

AWS Lambda runs a **handler** when an event arrives: an API request, S3 upload, queue message, schedule or direct invoke. You choose a runtime (Python, Node.js, Java, .NET, Ruby, or custom / container), memory (CPU scales with it) and timeout. The function runs with an **execution role**. **Function URLs** give a dedicated HTTPS endpoint without API Gateway.

KEY POINTS

- Code outside the handler runs once per execution environment: put clients there
- Environment variables configure the function (encrypted at rest)
- Synchronous, asynchronous and poll-based (event source mapping) invocations
- Arm64 (Graviton) is cheaper per GB-second

SECURITY CHECK

- ⚠ Least-privilege execution role per function
- ⚠ Function URLs: use `AWS_IAM` auth unless the endpoint must be public
- ⚠ Keep secrets in Secrets Manager, not environment variables

LIMITS & QUOTAS

- ≤ Timeout up to 15 minutes; memory 128 MB–10,240 MB
- ≤ Payload 6 MB synchronous; 1 MB asynchronous (raised from 256 KB in 2025)
- ≤ Zip package 50 MB direct upload / 250 MB unzipped; images up to 10 GB

HANDLER.PY

```python
import json, os, boto3
s3 = boto3.client("s3")                    # created once per environment
BUCKET = os.environ["BUCKET"]

def handler(event, context):
    body = json.loads(event.get("body") or "{}")
    key = f"orders/{body['id']}.json"
    s3.put_object(Bucket=BUCKET, Key=key, Body=json.dumps(body))
    return {"statusCode": 201, "body": json.dumps({"key": key})}
```

CODE EXPLAINED

- → The S3 client is reused across warm invocations
- → Arm64 and 512 MB keep cost low while giving enough CPU
- → `raw-in-base64-out` lets the CLI take a plain JSON payload
- → Logs go to the `/aws/lambda/<name>` log group

CREATE, INVOKE, EXPOSE

```
zip fn.zip handler.py
aws lambda create-function --function-name orders --runtime python3.13 \
    --architectures arm64 --handler handler.handler --zip-file fileb://fn.zip \
    --role $LAMBDA_ROLE --memory-size 512 --timeout 10 \
    --environment Variables={BUCKET=acme-orders}
aws lambda invoke --function-name orders --cli-binary-format raw-in-base64-out \
    --payload '{"body": "{\"id\": 42}"}' out.json && cat out.json
aws lambda update-function-code --function-name orders --zip-file fileb://fn.zip
aws lambda create-function-url-config --function-name orders --auth-type AWS_IAM
aws logs tail /aws/lambda/orders --follow
```

AZURE / GCP EQUIVALENT

- ↔ **Azure** Azure Functions, triggers and bindings (az-20)
- ↔ **GCP** Cloud Run (gcp-20), Cloud Run functions (gcp-21)

COST CHECK

- $ Per request + GB-second; generous monthly free tier
- $ Arm64 is about 20% cheaper per GB-second
- $ More memory can be cheaper if it shortens run time

INVOCATION TYPES

TYPE	EXAMPLES	RETRIES
Synchronous	API Gateway, function URL, invoke	Caller retries
Asynchronous	S3, SNS, EventBridge	2 retries, then DLQ / destination
Event source mapping	SQS, Kinesis, DynamoDB Streams	Until success / expiry

RUNTIMES

RUNTIME	IDENTIFIER
Python	python3.13
Node.js	nodejs22.x
Java	java21
.NET	dotnet8
Custom / Go / Rust	provided.al2023
Container	Image

DO & DON'T

- ✓ **Do** Initialize clients outside the handler
- ✓ **Do** Configure DLQs or on-failure destinations for async invokes
- ✗ **Don't** Use Lambda for long-running or always-busy workloads
- ✗ **Don't** Open function URLs with `NONE` auth for internal APIs

LOCAL TESTING

- ⌂ `sam local invoke` and `sam local start-api` run functions in Docker
- ⌂ Unit-test the handler as a plain function with fake events
- ⌂ LocalStack runs Lambda with other emulated services

PRODUCTION CHECKLIST

- ☐ Alarms on errors, throttles and duration
- ☐ Reserved concurrency to protect downstream systems
- ☐ Execution role least privilege; secrets from Secrets Manager

Lambda Operations

layers · versions & aliases · reserved / provisioned concurrency · cold starts · SnapStart · container images · event source mappings

RUNNING LAMBDA IN PRODUCTION

Versions freeze code and config; **aliases** (e.g. `live`) point at versions and can split traffic for canaries. **Reserved concurrency** caps (and guarantees) a function's share; **provisioned concurrency** keeps environments warm to remove **cold starts**. **SnapStart** snapshots initialized environments for Java, Python and .NET. **Layers** share libraries; **event source mappings** poll SQS, Kinesis and streams.

KEY POINTS

- Account concurrency is shared by all functions in a Region
- Alias routing configuration enables weighted canary releases
- Partial batch responses avoid retrying a whole SQS batch
- Lambda Powertools adds logging, tracing, idempotency helpers

LIMITS & QUOTAS

- ≤ Default 1,000 concurrent executions per Region (adjustable)
- ≤ Up to 5 layers per function; 250 MB unzipped total
- ≤ Each function scales by up to 1,000 environments every 10 seconds

VERSIONS, ALIASES, CONCURRENCY, QUEUES

```
V=$(aws lambda publish-version --function-name orders \
  --query Version --output text)
aws lambda create-alias --function-name orders --name live --function-version $V
# Canary: 10% of traffic to the new version
aws lambda update-alias --function-name orders --name live \
    --function-version $OLD \
    --routing-config "AdditionalVersionWeights={\"$V\"=0.1}"

aws lambda put-function-concurrency --function-name orders \
    --reserved-concurrent-executions 50
aws lambda put-provisioned-concurrency-config --function-name orders \
    --qualifier live --provisioned-concurrent-executions 5

aws lambda create-event-source-mapping --function-name orders \
    --event-source-arn arn:aws:sqs:us-east-1:111122223333:orders \
    --batch-size 10 --function-response-types ReportBatchItemFailures
aws lambda publish-layer-version --layer-name common \
    --zip-file fileb://layer.zip --compatible-runtimes python3.13
aws lambda update-function-configuration --function-name orders \
    --snap-start ApplyOn=PublishedVersions
```

CODE EXPLAINED

- → Callers invoke the `live` alias, not `$LATEST`
- → Weights shift 10% of invocations to the new version
- → Reserved concurrency 50 protects the database from overload
- → `ReportBatchItemFailures` retries only the failed messages

AZURE / GCP EQUIVALENT

- ↔ **Azure** App Service slots, Durable Functions, Functions plans (az-21)
- ↔ **GCP** Cloud Run revisions and jobs, Cloud Run functions (gcp-21)

COST CHECK

- $ Provisioned concurrency bills per hour even when idle
- $ SnapStart for Python / .NET bills for caching and restores; Java is free
- $ Reserved concurrency is free

CONCURRENCY CONTROLS

SETTING	DOES
`Reserved`	Max for this function (and guaranteed)
`Provisioned`	Pre-warmed environments
`Maximum concurrency (SQS ESM)`	Caps pollers per queue
`Account limit`	Shared Region pool

COLD START FIXES

FIX	EFFECT
`Smaller package / fewer deps`	Faster init
`SnapStart`	Restore from snapshot
`Provisioned concurrency`	Always warm
`Arm64 + right memory`	More CPU for init
`Avoid VPC unless needed`	Fewer network setup steps (now small)

DO & DON'T

- ✓ **Do** Deploy through aliases with canaries (CodeDeploy can automate)
- ✓ **Do** Use partial batch responses for SQS
- ✗ **Don't** Let one function consume the whole account concurrency
- ✗ **Don't** Pay for provisioned concurrency on low-traffic internal jobs

LOCAL TESTING

- ⌂ SAM accelerate (`sam sync --watch`) for fast dev loops
- ⌂ Lambda Powertools has test helpers
- ⌂ Replay sample SQS events locally with `sam local invoke -e`

PRODUCTION CHECKLIST

- ☐ Aliases + gradual deployments with automatic rollback alarms
- ☐ Reserved concurrency on critical / dangerous functions
- ☐ Powertools logging, tracing and idempotency

API Gateway

MANAGED APIS IN FRONT OF YOUR CODE

API Gateway exposes HTTPS APIs backed by Lambda, HTTP endpoints or AWS services. **HTTP APIs** are cheaper, faster and simpler, with built-in JWT authorizers and CORS. **REST APIs** add request validation, usage plans with API keys, caching, WAF and private endpoints. **Stages** deploy versions; throttling protects back ends.

KEY POINTS

- WebSocket APIs handle two-way connections
- Custom domains map to stages with ACM certificates
- Usage plans + API keys meter clients (keys are not authentication)
- Default endpoint can be disabled when using a custom domain

SECURITY CHECK

- ⚠ Authenticate with JWT (Cognito / OIDC) or IAM authorizers
- ⚠ Attach AWS WAF to REST APIs (or front HTTP APIs with CloudFront + WAF)
- ⚠ Disable the default execute-api endpoint when a custom domain is used

HTTP API WITH LAMBDA AND JWT AUTH

```
API=$(aws apigatewayv2 create-api --name shop --protocol-type HTTP \
  --cors-configuration AllowOrigins=https://shop.acme.com,AllowMethods=POST \
  --query ApiId --output text)
INT=$(aws apigatewayv2 create-integration --api-id $API \
  --integration-type AWS_PROXY --integration-uri $FN_ARN \
  --payload-format-version 2.0 --query IntegrationId --output text)
AUTH=$(aws apigatewayv2 create-authorizer --api-id $API --name jwt \
  --authorizer-type JWT --identity-source '$request.header.Authorization' \
  --jwt-configuration Audience=shop-api,Issuer=$COGNITO_ISSUER_URL \
  --query AuthorizerId --output text)
aws apigatewayv2 create-route --api-id $API --route-key "POST /orders" \
    --target integrations/$INT --authorization-type JWT --authorizer-id $AUTH
aws apigatewayv2 create-stage --api-id $API --stage-name '$default' \
    --auto-deploy \
    --default-route-settings ThrottlingBurstLimit=100,ThrottlingRateLimit=50
aws lambda add-permission --function-name orders --statement-id apigw \
    --action lambda:InvokeFunction --principal apigateway.amazonaws.com \
    --source-arn "arn:aws:execute-api:us-east-1:111122223333:$API/*"
```

CODE EXPLAINED

- → HTTP APIs set CORS in one flag
- → Payload format 2.0 is the simpler event shape for Lambda
- → The JWT authorizer validates Cognito tokens before Lambda runs
- → Lambda needs a resource policy allowing API Gateway to invoke it

AZURE / GCP EQUIVALENT

- ↔ **Azure** API Management (az-22)
- ↔ **GCP** API Gateway, Apigee (gcp-22)

COST CHECK

- $ HTTP APIs cost much less per million requests than REST APIs
- $ REST API caching bills hourly by cache size
- $ Throttle to stop runaway clients from running up Lambda costs

HTTP API VS REST API

FEATURE	HTTP API	REST API
Price	Lower	Higher
JWT authorizer	Built in	Via Lambda / Cognito
Usage plans / API keys	No	Yes
Caching	No	Yes
WAF	No (use CloudFront)	Yes
Private API	No	Yes

AUTHORIZERS

TYPE	USE
JWT	OIDC / Cognito tokens (HTTP API)
Cognito user pool	REST API
Lambda	Custom logic
IAM (SigV4)	Service-to-service

DO & DON'T

- ✓ **Do** Start with HTTP APIs unless you need REST features
- ✓ **Do** Set throttling on every stage
- ✗ **Don't** Rely on API keys as authentication
- ✗ **Don't** Return stack traces from integrations

LOCAL TESTING

- ⌂ `sam local start-api` runs routes against local Lambdas
- ⌂ Test JWTs with a local OIDC provider or Cognito test users
- ⌂ Validate OpenAPI definitions with Spectral

PRODUCTION CHECKLIST

- ☐ Authorizer on every non-public route
- ☐ Access logging and throttling configured
- ☐ Custom domain with ACM; default endpoint disabled

EventBridge & Step Functions

event buses · rules & event patterns · EventBridge Scheduler · Step Functions state machines · Amazon States Language · Standard vs Express

EVENTS AND WORKFLOWS

EventBridge routes events from AWS services, SaaS partners and your apps to targets by matching **event patterns** on **rules**; **Scheduler** runs one-time or recurring invocations. **Step Functions** orchestrates multi-step workflows defined in the **Amazon States Language** (ASL): tasks, choices, retries, parallel branches, maps and waits, with direct integrations to 200+ services.

KEY POINTS

- The default bus receives AWS service events
- Archive and replay events for recovery or testing
- Standard workflows: durable, up to a year; Express: high-volume, up to 5 minutes
- Retries and catches are declared per state

RULE, SCHEDULE, STATE MACHINE

```
aws events put-rule --name order-created --event-bus-name shop \
    --event-pattern '{"source":["acme.orders"],"detail-type":["OrderCreated"],
      "detail":{"total":[{"numeric":[">",100]}]}}'
aws events put-targets --rule order-created --event-bus-name shop \
    --targets Id=sfn,Arn=$SFN_ARN,RoleArn=$EVENTS_ROLE
aws events put-events --entries file://order-created.json

aws scheduler create-schedule --name nightly-report \
    --schedule-expression "cron(0 2 * * ? *)" --flexible-time-window Mode=OFF \
    --target "{\"Arn\":\"$FN_ARN\",\"RoleArn\":\"$SCHED_ROLE\"}"
```

CODE EXPLAINED

- → The pattern matches large orders only (numeric filter)
- → EventBridge needs a role to start the state machine
- → Scheduler cron has six fields including year
- → Step Functions retries the charge step with backoff, then queues shipping

AMAZON STATES LANGUAGE (EXCERPT)

```
{ "StartAt": "Charge",
  "States": {
    "Charge": { "Type": "Task", "Resource": "arn:aws:states:::lambda:invoke",
      "Parameters": { "FunctionName": "charge", "Payload.$": "$" },
      "Retry": [{ "ErrorEquals": ["States.TaskFailed"], "MaxAttempts": 3,
                  "BackoffRate": 2 }],
      "Next": "Ship" },
    "Ship": { "Type": "Task", "End": true,
      "Resource": "arn:aws:states:::sqs:sendMessage",
      "Parameters": { "QueueUrl": "${ShipQueueUrl}", "MessageBody.$": "$" } }
  } }
```

AZURE / GCP EQUIVALENT

- ↔ **Azure** Event Grid, Logic Apps, Durable Functions (az-23)
- ↔ **GCP** Eventarc, Workflows, Cloud Scheduler (gcp-23)

COST CHECK

- $ EventBridge: per million events published (AWS service events on the default bus are free)
- $ Step Functions Standard: per state transition; Express: per request + duration
- $ Scheduler has a large monthly free tier

STANDARD VS EXPRESS

FEATURE	STANDARD	EXPRESS
Max duration	1 year	5 minutes
Execution model	Exactly-once	At-least-once
History	Full, 90 days	CloudWatch Logs
Price model	Per transition	Per request + duration
Use	Business processes	High-volume event processing

ASL STATE TYPES

TYPE	DOES
Task	Do work
Choice	Branch
Parallel	Branches at once
Map	For each item
Wait	Delay
Pass / Succeed / Fail	Control flow

DO & DON'T

- ✓ **Do** Use a custom bus per domain with schemas
- ✓ **Do** Put retries and catches in the state machine, not the code
- ✗ **Don't** Build long synchronous Lambda chains instead of a workflow
- ✗ **Don't** Match events too broadly and trigger targets by accident

LOCAL TESTING

- ⌂ Step Functions Local (Docker) runs ASL locally
- ⌂ EventBridge `test-event-pattern` checks patterns offline
- ⌂ Workflow Studio designs state machines visually

PRODUCTION CHECKLIST

- ☐ DLQs on rule targets; archives for replay
- ☐ Alarms on failed executions and throttled targets
- ☐ State machines defined in IaC with versions / aliases

SQS & SNS

QUEUES AND TOPICS

SQS queues hold messages until a consumer processes and deletes them. **Standard** queues give nearly unlimited throughput with at-least-once, best-effort ordering; **FIFO** queues give ordering per message group and deduplication. The **visibility timeout** hides a message while it is processed. **SNS** topics push each message to many subscribers (SQS, Lambda, HTTP, email): the **fan-out** pattern.

KEY POINTS

- Long polling (`WaitTimeSeconds` up to 20) cuts empty receives
- A redrive policy moves repeatedly failing messages to a DLQ
- SNS subscription filter policies route by message attributes
- Set visibility timeout to at least 6× the Lambda timeout for Lambda consumers

LIMITS & QUOTAS

- ≤ SQS message up to 1 MiB (raised from 256 KB in 2025)
- ≤ Retention 1 minute–14 days (default 4 days)
- ≤ Visibility timeout up to 12 hours

QUEUE, DLQ, TOPIC FAN-OUT

```
DLQ=$(aws sqs create-queue --queue-name orders-dlq \
  --query QueueUrl --output text)
DLQ_ARN=$(aws sqs get-queue-attributes --queue-url $DLQ \
  --attribute-names QueueArn --query Attributes.QueueArn --output text)
Q=$(aws sqs create-queue --queue-name orders \
  --attributes file://orders-attrs.json --query QueueUrl --output text)
# orders-attrs.json:
# {"VisibilityTimeout": "60", "ReceiveMessageWaitTimeSeconds": "20",
#  "RedrivePolicy": "{\"deadLetterTargetArn\": \"<dlq-arn>\",
#                    \"maxReceiveCount\": \"5\"}"}

aws sqs send-message --queue-url $Q --message-body '{"id": 42}'
aws sqs receive-message --queue-url $Q --max-number-of-messages 10
aws sqs delete-message --queue-url $Q --receipt-handle <handle>

T=$(aws sns create-topic --name order-events --query TopicArn --output text)
aws sns subscribe --topic-arn $T --protocol sqs --notification-endpoint $Q_ARN \
    --attributes file://sub-attrs.json   # filter policy: region = eu
aws sns publish --topic-arn $T --message '{"id": 42}' \
    --message-attributes '{"region":{"DataType":"String","StringValue":"eu"}}'
aws sqs start-message-move-task --source-arn $DLQ_ARN   # redrive back
```

CODE EXPLAINED

- → After 5 failed receives a message moves to the DLQ
- → Delete with the receipt handle after successful processing
- → The SQS subscription only gets EU messages (filter policy)
- → `start-message-move-task` redrives DLQ messages to the source queue

AZURE / GCP EQUIVALENT

- ↔ **Azure** Service Bus queues / topics, Storage Queues (az-24)
- ↔ **GCP** Pub/Sub topics and subscriptions (gcp-24)

COST CHECK

- $ Per million requests; first million each month free (SQS and SNS)
- $ Long polling and batching (up to 10 messages) cut request counts
- $ SNS to HTTP / SMS / email has delivery charges

STANDARD VS FIFO

FEATURE	STANDARD	FIFO
Throughput	Nearly unlimited	High with high-throughput mode
Ordering	Best effort	Per message group
Delivery	At least once	Exactly-once processing (dedup)
Name	any	Must end in .fifo

PATTERNS

PATTERN	BUILD
Work queue	SQS + consumers
Fan-out	SNS → many SQS
Ordered processing	SQS FIFO + group IDs
Event routing by content	EventBridge (sheet 23)
Retry with delay	Visibility timeout / delay queues

DO & DON'T

- ✓ **Do** Make consumers idempotent and delete only after success
- ✓ **Do** Alarm on DLQ depth and oldest message age
- ✗ **Don't** Set visibility timeout shorter than processing time
- ✗ **Don't** Send large blobs in messages (store in S3, send a pointer)

LOCAL TESTING

- ⌂ LocalStack emulates SQS and SNS
- ⌂ ElasticMQ is a lightweight SQS-compatible server
- ⌂ moto mocks SQS / SNS in Python tests

PRODUCTION CHECKLIST

- ☐ DLQs with alarms and a redrive process
- ☐ Encryption (SSE-SQS / KMS) on queues and topics
- ☐ Queue policies restricting senders

Elastic Container Registry

aws ecr create-repository · get-login-password | docker login · tag & push · lifecycle policies · image scanning · immutable tags

PRIVATE CONTAINER IMAGES ON AWS

Amazon ECR stores container images and OCI artifacts in private repositories (and offers a public gallery). Docker logs in with a short-lived token from `get-login-password`. **Immutable tags** prevent overwrites, **lifecycle policies** expire old images, and **scanning** (basic or enhanced with Inspector) reports vulnerabilities. Pull-through cache rules mirror public registries.

KEY POINTS

- Registry URL: `<account>.dkr.ecr.<region>.amazonaws.com`
- ECS, EKS, Lambda and App Runner pull with IAM roles
- Replication copies images across Regions and accounts
- Pull-through cache avoids Docker Hub rate limits

SECURITY CHECK

- ⚠ Enable tag immutability on release repositories
- ⚠ Enhanced scanning (Inspector) with alerts on critical findings
- ⚠ Repository policies limiting who can push

LIMITS & QUOTAS

- ≤ Repositories and images per repository have adjustable quotas (Service Quotas)
- ≤ Lifecycle policies: up to 50 rules each
- ≤ API rate limits per account and Region apply to large fleets

CREATE, PUSH, EXPIRE

```
aws ecr create-repository --repository-name shop-api \
    --image-tag-mutability IMMUTABLE \
    --image-scanning-configuration scanOnPush=true \
    --encryption-configuration encryptionType=KMS
REG=111122223333.dkr.ecr.us-east-1.amazonaws.com
aws ecr get-login-password | docker login --username AWS --password-stdin $REG

docker build -t $REG/shop-api:1.4.0 .
docker push $REG/shop-api:1.4.0
aws ecr describe-images --repository-name shop-api \
    --query "imageDetails[].[imageTags[0],imagePushedAt]" --output table
aws ecr describe-image-scan-findings --repository-name shop-api \
    --image-id imageTag=1.4.0 --query "imageScanFindings.findingSeverityCounts"

aws ecr put-lifecycle-policy --repository-name shop-api \
    --lifecycle-policy-text '{"rules":[{"rulePriority":1,
      "selection":{"tagStatus":"untagged","countType":"sinceImagePushed",
        "countUnit":"days","countNumber":14},"action":{"type":"expire"}}]}'
aws ecr create-pull-through-cache-rule --ecr-repository-prefix dockerhub \
    --upstream-registry-url registry-1.docker.io --credential-arn <secret-arn>
```

CODE EXPLAINED

- → Immutable tags stop a release tag from being overwritten
- → The login token lasts 12 hours
- → Scan findings summarize vulnerabilities by severity
- → The lifecycle rule expires untagged images after 14 days

AZURE / GCP EQUIVALENT

- ↔ **Azure** Azure Container Registry (az-25)
- ↔ **GCP** Artifact Registry (gcp-25)

COST CHECK

- $ Storage per GB-month plus data transfer out
- $ Pulls inside the same Region are free (other than NAT / endpoint charges)
- $ Lifecycle policies keep storage from growing forever

REPOSITORY SETTINGS

SETTING	USE
Tag immutability	Release integrity
Scan on push / enhanced	Vulnerabilities
KMS encryption	Key control
Lifecycle policy	Expire old images
Replication	Multi-Region / account
Repository policy	Cross-account access

PULL PATHS

CONSUMER	AUTH
ECS / Fargate	Task execution role
EKS nodes	Node role
Lambda	Function service
Laptop / CI	get-login-password

DO & DON'T

- ✓ **Do** Tag images with version + commit SHA
- ✓ **Do** Add VPC endpoints (ecr.api, ecr.dkr, S3) for private pulls
- ✗ **Don't** Deploy `latest`
- ✗ **Don't** Keep every CI build forever

LOCAL TESTING

- ⌂ `docker run -d -p 5000:5000 registry:2` for a local registry
- ⌂ Trivy scans images before push
- ⌂ LocalStack Pro emulates ECR

PRODUCTION CHECKLIST

- ☐ Immutable tags + enhanced scanning on release repos
- ☐ VPC endpoints for private pulls
- ☐ Replication for DR Regions where needed

ECS & Fargate

clusters · task definitions · services · Fargate vs EC2 launch type · awsvpc networking · task roles · ECS Exec

CONTAINERS THE AWS WAY

Amazon ECS runs containers from **task definitions** (image, CPU, memory, ports, env, logging, roles). A **service** keeps a desired number of tasks running behind a load balancer with rolling or blue/green deploys. **Fargate** runs tasks without servers; the EC2 launch type uses your instances. Each task gets its own ENI (`awsvpc`), a **task role** for app permissions and an execution role for pulling images and logs.

KEY POINTS

- Task role = what the app can do; execution role = what ECS needs to start it
- Secrets come from Secrets Manager / Parameter Store via `secrets`
- Capacity providers mix Fargate and Fargate Spot
- ECS Exec opens a shell in a running container

TASK DEFINITION (EXCERPT)

```
{ "family": "shop-api", "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"], "cpu": "512", "memory": "1024",
  "runtimePlatform": { "cpuArchitecture": "ARM64" },
  "taskRoleArn": "arn:aws:iam::111122223333:role/shop-api-task",
  "executionRoleArn": "arn:aws:iam::111122223333:role/ecsTaskExecutionRole",
  "containerDefinitions": [{ "name": "api", "essential": true,
    "image": "111122223333.dkr.ecr.us-east-1.amazonaws.com/shop-api:1.4.0",
    "portMappings": [{ "containerPort": 8080 }],
    "readonlyRootFilesystem": true, "user": "1000",
    "secrets": [{ "name": "DB_PASSWORD",
      "valueFrom": "arn:aws:secretsmanager:us-east-1:1111:secret:prod/db" }],
    "logConfiguration": { "logDriver": "awslogs", "options": {
      "awslogs-group": "/ecs/shop-api", "awslogs-region": "us-east-1",
      "awslogs-stream-prefix": "api" } } }] }
```

CODE EXPLAINED

→ Fargate tasks declare CPU and memory at the task level

→ ARM64 runs on Graviton for lower cost

→ The `secrets` block injects the value at start using the execution role

→ ECS Exec needs `--enable-execute-command` and SSM permissions

AZURE / GCP EQUIVALENT

↔ **Azure** Azure Container Apps (az-26)

↔ **GCP** Cloud Run (gcp-20); GKE for Kubernetes (gcp-26)

COST CHECK

$ Fargate: per vCPU-second and GB-second while tasks run

$ Fargate Spot: up to ~70% off, interruptible

$ ECS control plane is free; EC2 launch type pays for instances

CLUSTER, SERVICE, EXEC

```
aws ecs create-cluster --cluster-name shop \
    --capacity-providers FARGATE FARGATE_SPOT
aws ecs register-task-definition --cli-input-json file://taskdef.json
aws ecs create-service --cluster shop --service-name api \
    --task-definition shop-api --desired-count 2 --launch-type FARGATE \
    --network-configuration file://awsvpc.json \
    --load-balancers targetGroupArn=$TG,containerName=api,containerPort=8080 \
    --enable-execute-command
aws ecs execute-command --cluster shop --task <task-id> --container api \
    --interactive --command "/bin/sh"
```

SECURITY CHECK

⚠ Separate task and execution roles with least privilege

⚠ Run tasks in private subnets without public IPs

⚠ Read-only root file system and non-root user in task definitions

LIMITS & QUOTAS

≤ Fargate On-Demand vCPU quota per Region (adjustable)

≤ Fargate task size up to 16 vCPU / 120 GB

≤ 10 containers per task definition

LAUNCH TYPES

ITEM	FARGATE	EC2
Servers	None to manage	Your instances
GPUs / special hardware	Limited	Yes
Scale to zero	Desired 0	ASG to 0
Price model	Per task resources	Per instance

DEPLOYMENT OPTIONS

OPTION	DOES
Rolling update	Replace tasks in batches
Blue/green (built-in or CodeDeploy)	Shift traffic between task sets
Circuit breaker	Auto-rollback failed deploys
Service Connect	Service-to-service discovery

DO & DON'T

✓ **Do** Enable the deployment circuit breaker with rollback

✓ **Do** Use Fargate Spot for workers and dev

✗ **Don't** Put secrets in plain environment variables

✗ **Don't** Give the task role the execution role's permissions (or vice versa) without need

LOCAL TESTING

⌂ `docker run` the same image with the same env

⌂ Docker Compose mimics multi-container tasks

⌂ AWS Copilot CLI and `ecs-cli` alternatives scaffold services

PRODUCTION CHECKLIST

☐ Private subnets, circuit breaker, health checks

☐ Container Insights and log retention set

☐ Autoscaling on CPU / requests per target

EKS

MANAGED KUBERNETES ON AWS

Amazon EKS runs the Kubernetes control plane across AZs. Workers come from **managed node groups**, Fargate profiles, Karpenter, or **EKS Auto Mode**, which manages compute, networking and storage components for you. **EKS Pod Identity** gives pods IAM roles. **Add-ons** (VPC CNI, CoreDNS, kube-proxy, EBS CSI) are versioned and managed.

KEY POINTS

- Access entries map IAM principals to Kubernetes permissions
- The VPC CNI gives pods real VPC IP addresses
- Upgrade the control plane, then add-ons and nodes
- See the Kubernetes set for kubectl and manifests

LIMITS & QUOTAS

- ≤ 100 clusters per Region by default
- ≤ 30 managed node groups per cluster; 450 nodes per group
- ≤ 10 Fargate profiles per cluster

CLUSTER, ACCESS, POD IDENTITY

```
eksctl create cluster --name shop --region us-east-1 --version 1.34 \
    --enable-auto-mode
# or managed node groups:
# eksctl create cluster --name shop --nodegroup-name ng1 \
#   --node-type m7g.large --nodes 3 --nodes-min 3 --nodes-max 10

aws eks update-kubeconfig --name shop --region us-east-1
kubectl get nodes

aws eks create-access-entry --cluster-name shop \
    --principal-arn arn:aws:iam::111122223333:role/platform-admins
aws eks associate-access-policy --cluster-name shop \
    --principal-arn arn:aws:iam::111122223333:role/platform-admins \
    --policy-arn $EKS_ADMIN_POLICY_ARN \
    --access-scope type=cluster

aws eks create-pod-identity-association --cluster-name shop \
    --namespace shop --service-account api \
    --role-arn arn:aws:iam::111122223333:role/shop-api-pod
aws eks list-addons --cluster-name shop
aws eks update-cluster-version --name shop --kubernetes-version <next>
```

CODE EXPLAINED

- → Auto Mode provisions and scales nodes automatically
- → `update-kubeconfig` writes a context that authenticates with IAM
- → Access entries replace editing the aws-auth ConfigMap
- → Pods using service account `shop/api` get the role's credentials

AZURE / GCP EQUIVALENT

- ↔ **Azure** AKS, AKS Automatic, workload identity (az-27)
- ↔ **GCP** GKE Autopilot / Standard, Workload Identity Federation (gcp-26, gcp-27)

COST CHECK

- $ Cluster fee per hour (higher in extended support)
- $ Auto Mode adds a per-instance management fee on top of EC2
- $ Spot nodes and Karpenter consolidation cut node cost

COMPUTE OPTIONS

OPTION	USE
EKS Auto Mode	AWS manages nodes and core add-ons
Managed node groups	You pick types; AWS manages lifecycle
Karpenter	Fast, flexible autoscaling
Fargate profiles	Serverless pods
Self-managed nodes	Full control

CORE ADD-ONS

ADD-ON	DOES
Amazon VPC CNI	Pod networking
CoreDNS	Cluster DNS
kube-proxy	Service routing
EBS CSI driver	Persistent volumes
Pod Identity agent	IAM for pods

DO & DON'T

- ✓ **Do** Use Pod Identity (or IRSA) for every workload needing AWS access
- ✓ **Do** Stay within standard support versions
- ✗ **Don't** Run apps with the node role's permissions
- ✗ **Don't** Hand-edit aws-auth when access entries exist

LOCAL TESTING

- ⌂ kind / k3d / minikube run the same manifests
- ⌂ eksctl dry-run prints the CloudFormation it will create
- ⌂ Telepresence / mirrord bridge local code to a cluster

PRODUCTION CHECKLIST

- ☐ Control plane logging and Container Insights
- ☐ Upgrades planned before end of standard support
- ☐ Pod Identity, network policies, and private endpoint access

S3 Basics

OBJECT STORAGE ON AWS

Amazon S3 stores objects under **keys** in **buckets**. Bucket names are globally unique; a bucket lives in one Region. The high-level `aws s3` commands copy and sync files (with automatic multipart upload); `aws s3api` exposes every API. **Storage classes** trade storage price for retrieval price and speed, and **Intelligent-Tiering** moves objects automatically.

KEY POINTS

- "Folders" are key prefixes (`2026/11/photo.jpg`)
- Strong read-after-write consistency for all operations
- New buckets block public access and use bucket-owner-enforced ownership by default
- S3 Express One Zone offers single-digit millisecond access in one AZ

SECURITY CHECK

- ⚠ Keep Block Public Access on at account and bucket level
- ⚠ Require TLS with a bucket policy (`aws:SecureTransport`)
- ⚠ Use SSE-KMS with a customer managed key for sensitive data

LIMITS & QUOTAS

- ≤ Objects up to 5 TB; single PUT up to 5 GB (use multipart beyond)
- ≤ Buckets per account: 10,000 by default (adjustable)
- ≤ At least 3,500 writes / 5,500 reads per second per prefix

EVERYDAY S3

```
aws s3 mb s3://acme-media-prod --region us-east-1
aws s3 cp photo.jpg s3://acme-media-prod/2026/11/photo.jpg
aws s3 cp ./images s3://acme-media-prod/images/ --recursive \
    --exclude "*" --include "*.jpg"
aws s3 sync ./site s3://acme-site --delete
aws s3 ls s3://acme-media-prod/2026/ --recursive --human-readable --summarize
aws s3 rm s3://acme-media-prod/tmp/ --recursive

aws s3 cp backup.tar s3://acme-backup/ --storage-class GLACIER_IR
aws s3api head-object --bucket acme-media-prod --key 2026/11/photo.jpg
aws s3api put-object-tagging --bucket acme-media-prod \
    --key 2026/11/photo.jpg --tagging 'TagSet=[{Key=team,Value=web}]'
aws s3api list-objects-v2 --bucket acme-media-prod --prefix 2026/ \
    --query "Contents[?Size>\`1000000\`].Key"
```

CODE EXPLAINED

- → `cp --recursive` with include / exclude filters copies by pattern
- → `sync --delete` mirrors a folder exactly
- → GLACIER_IR keeps millisecond access at archive-like storage prices
- → JMESPath filters objects larger than 1 MB

AZURE / GCP EQUIVALENT

- ↔ **Azure** Blob Storage: storage accounts, containers, tiers (az-28)
- ↔ **GCP** Cloud Storage: buckets, classes, Autoclass (gcp-28)

PYTHON

```
import boto3
s3 = boto3.client("s3")
s3.upload_file("big.zip", "acme-backup", "2026/big.zip")   # multipart if large
obj = s3.get_object(Bucket="acme-media-prod", Key="2026/11/photo.jpg")
data = obj["Body"].read()
```

COST CHECK

- $ Storage per GB-month by class, plus requests, retrieval and egress
- $ Minimum storage durations and retrieval fees on IA / Glacier classes
- $ Incomplete multipart uploads bill until aborted (add a lifecycle rule)

STORAGE CLASSES

CLASS	USE	MIN DAYS
Standard	Frequent	–
Intelligent-Tiering	Unknown patterns	–
Standard-IA / One Zone-IA	Monthly access	30
Glacier Instant Retrieval	Quarterly, ms access	90
Glacier Flexible Retrieval	Minutes-hours	90
Glacier Deep Archive	Hours, cheapest	180

S3 VS S3API

COMMAND	USE
aws s3 cp / sync / mv / rm	Files and folders
aws s3 presign	Temporary URLs
aws s3api put-bucket-*	Bucket settings
aws s3api get / put-object	Single-object control

DO & DON'T

- ✓ **Do** Use prefixes that group data by lifecycle (logs/, tmp/)
- ✓ **Do** Add a lifecycle rule to abort incomplete multipart uploads
- ✗ **Don't** Make buckets public to serve websites (use CloudFront + OAC)
- ✗ **Don't** List huge buckets repeatedly (use S3 Inventory)

LOCAL TESTING

- ⌂ LocalStack and MinIO emulate S3 APIs
- ⌂ moto mocks S3 in Python tests
- ⌂ S3 Mountpoint mounts a bucket as a file system for experiments

PRODUCTION CHECKLIST

- □ Block Public Access, TLS-only policy, SSE-KMS where needed
- □ Versioning and lifecycle rules (sheet 29)
- □ Server access logs or CloudTrail data events for audit

S3 Security & Features

Block Public Access · bucket policies · presigned URLs · versioning · lifecycle rules · SSE-S3 / SSE-KMS · static website hosting

PROTECTING AND MANAGING S3 DATA

Control access with IAM and **bucket policies**, keep **Block Public Access** on, and share single objects with time-limited **presigned URLs**. **Versioning** keeps every version of an object; **Object Lock** adds write-once retention. **Lifecycle rules** transition and expire objects. Encryption at rest is automatic (SSE-S3); **SSE-KMS** adds key control and audit.

KEY POINTS

- ACLs are disabled by default (bucket owner enforced); use policies
- Presigned URLs carry the signer's permissions until they expire
- MFA delete and Object Lock protect against destructive deletes
- Replication (same or cross Region) needs versioning

SECURITY CHECK

- ⚠ Block Public Access at the account level
- ⚠ Bucket policy denying non-TLS and wrong-key uploads
- ⚠ Versioning + Object Lock or AWS Backup for critical data

POLICIES, PRESIGNING, VERSIONING, LIFECYCLE

```
aws s3api put-public-access-block --bucket acme-media-prod \
    --public-access-block-configuration BlockPublicAcls=true,\
IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true
aws s3api put-bucket-policy --bucket acme-media-prod --policy file://tls.json
aws s3api put-bucket-encryption --bucket acme-media-prod \
    --server-side-encryption-configuration file://sse-kms.json
aws s3api put-bucket-versioning --bucket acme-media-prod \
    --versioning-configuration Status=Enabled
aws s3 presign s3://acme-media-prod/2026/11/photo.jpg --expires-in 900
aws s3api put-bucket-lifecycle-configuration --bucket acme-logs \
    --lifecycle-configuration file://lifecycle.json
```

LIFECYCLE.JSON

```
{ "Rules": [{ "ID": "logs", "Status": "Enabled",
    "Filter": { "Prefix": "logs/" },
    "Transitions": [{ "Days": 30, "StorageClass": "STANDARD_IA" },
                    { "Days": 180, "StorageClass": "GLACIER" }],
    "Expiration": { "Days": 730 },
    "NoncurrentVersionExpiration": { "NoncurrentDays": 30 },
    "AbortIncompleteMultipartUpload": { "DaysAfterInitiation": 7 } }] }
```

CODE EXPLAINED

- → All four Block Public Access switches on
- → Default SSE-KMS with an S3 Bucket Key cuts KMS request costs
- → The presigned URL works for 15 minutes
- → Lifecycle tiers logs, expires old ones, and cleans versions and uploads

AZURE / GCP EQUIVALENT

- ↔ **Azure** Blob data roles, user delegation SAS, versioning, lifecycle (az-29)
- ↔ **GCP** Uniform access, signed URLs, versioning, lifecycle (gcp-29)

COST CHECK

- $ Old versions bill as normal storage: expire noncurrent versions
- $ S3 Bucket Keys reduce SSE-KMS request charges
- $ Lifecycle transitions bill per request; tiny objects may not be worth moving

ACCESS CONTROLS

CONTROL	USE
IAM policy	Your principals
Bucket policy	Cross-account, conditions
Block Public Access	Guardrail against exposure
Access points	Per-app policies at scale
Presigned URL	Temporary object access
ACLs	Legacy (disabled by default)

DATA PROTECTION

FEATURE	PROTECTS AGAINST
Versioning	Overwrites / deletes
Object Lock	Tampering, ransomware
Replication	Region loss
AWS Backup for S3	Point-in-time recovery
MFA delete	Unauthorized version deletes

DO & DON'T

- ✓ **Do** Serve sites through CloudFront with OAC
- ✓ **Do** Expire noncurrent versions
- ✗ **Don't** Hand out long-lived presigned URLs
- ✗ **Don't** Disable Block Public Access to fix an access problem

LOCAL TESTING

- ⌂ LocalStack supports bucket policies and presigning
- ⌂ IAM Access Analyzer for S3 flags public or shared buckets
- ⌂ Validate lifecycle JSON in a sandbox bucket

PRODUCTION CHECKLIST

- ☐ Account-level Block Public Access
- ☐ Versioning + replication or Backup for critical buckets
- ☐ Macie or Access Analyzer reviews for sensitive / shared data

EBS, EFS & AWS Backup

BLOCK, FILE AND BACKUP

EBS volumes are block storage attached to one instance in one AZ (multi-attach for some io2 cases). **EFS** is an elastic NFS file system many instances, containers and Lambdas can mount across AZs. **FSx** offers Windows, Lustre, NetApp ONTAP and OpenZFS file systems. **AWS Backup** centralizes backup plans across EBS, EFS, RDS, DynamoDB, S3 and more.

KEY POINTS

- EBS snapshots are incremental and can be copied across Regions
- EFS storage classes (Standard, IA, Archive) with lifecycle management
- EFS access points enforce a path and POSIX user per app
- Backup vault lock makes backups immutable

EFS MOUNT AND BACKUP PLAN

```
FS=$(aws efs create-file-system --encrypted --throughput-mode elastic \
  --tags Key=Name,Value=shared --query FileSystemId --output text)
aws efs create-mount-target --file-system-id $FS --subnet-id $APP_A \
    --security-groups $EFS_SG
aws efs put-lifecycle-configuration --file-system-id $FS \
    --lifecycle-policies TransitionToIA=AFTER_30_DAYS
sudo mount -t efs -o tls $FS:/ /mnt/shared        # amazon-efs-utils

aws backup create-backup-vault --backup-vault-name prod-vault
aws backup create-backup-plan --backup-plan '{"BackupPlanName":"daily",
  "Rules":[{"RuleName":"daily","TargetBackupVaultName":"prod-vault",
    "ScheduleExpression":"cron(0 5 * * ? *)",
    "Lifecycle":{"DeleteAfterDays":35}}]}'
aws backup create-backup-selection --backup-plan-id <plan-id> \
    --backup-selection '{"SelectionName":"tagged","IamRoleArn":"<role-arn>",
      "ListOfTags":[{"ConditionType":"STRINGEQUALS",
        "ConditionKey":"backup","ConditionValue":"daily"}]}'
```

CODE EXPLAINED

- → Elastic throughput scales with demand automatically
- → One mount target per AZ lets instances mount locally
- → The EFS mount helper encrypts traffic in transit with `tls`
- → Backup selects resources by tag, so new resources join automatically

AZURE / GCP EQUIVALENT

- ↔ **Azure** managed disks, Azure Files, Azure Backup (az-30)
- ↔ **GCP** Persistent Disk / Hyperdisk, Filestore, Backup and DR (gcp-30)

COST CHECK

- $ EBS bills provisioned GB (and IOPS / throughput above gp3 baseline)
- $ EFS bills stored GB by class plus elastic throughput
- $ Backup bills per GB stored in vaults

WHICH STORAGE

NEED	USE
Objects over HTTPS	S3
Boot / database disk	EBS
Shared Linux files	EFS
Windows SMB	FSx for Windows
HPC scratch	FSx for Lustre

EBS TYPES

TYPE	USE
gp3	Default SSD, tunable
io2 Block Express	Highest IOPS
st1	Throughput HDD
sc1	Cold HDD

DO & DON'T

- ✓ **Do** Back up by tag with AWS Backup
- ✓ **Do** Use EFS access points per application
- ✗ **Don't** Use EFS for database data files
- ✗ **Don't** Keep snapshots forever without lifecycle

LOCAL TESTING

- ⌂ Practice NFS mounts with a local NFS server container
- ⌂ Restore drills from AWS Backup into a scratch account
- ⌂ fio benchmarks EBS settings on a test instance

PRODUCTION CHECKLIST

- ☐ Backup plans with vault lock for critical data
- ☐ Cross-Region / cross-account backup copies
- ☐ Restores tested regularly

RDS & Aurora

MANAGED RELATIONAL DATABASES

Amazon RDS runs PostgreSQL, MySQL, MariaDB, SQL Server, Oracle and Db2 with automated backups, patching and **Multi-AZ** failover. **Aurora** is AWS's cloud-native PostgreSQL / MySQL with storage replicated across three AZs, fast replicas and **Serverless v2** autoscaling. **RDS Proxy** pools connections for Lambda and spiky apps.

KEY POINTS

- Multi-AZ DB clusters (two readable standbys) are an option for MySQL / PostgreSQL
- Point-in-time restore creates a new instance
- Parameter groups hold engine settings
- IAM database authentication avoids passwords for apps

SECURITY CHECK

- ⚠ No public accessibility; private subnets only
- ⚠ Store master password in Secrets Manager (`--manage-master-user-password`)
- ⚠ Encryption at rest with KMS; enforce TLS (`rds.force_ssl`)

LIMITS & QUOTAS

- ≤ Backup retention 1–35 days for PITR
- ≤ Storage up to 64 TiB (most RDS engines); Aurora up to 128 TiB
- ≤ 40 DB instances per Region by default

RDS POSTGRESQL AND AURORA SERVERLESS V2

```
aws rds create-db-subnet-group --db-subnet-group-name shop-db \
    --db-subnet-group-description shop --subnet-ids $DATA_A $DATA_B $DATA_C
aws rds create-db-instance --db-instance-identifier shop-pg \
    --engine postgres --engine-version 17 --db-instance-class db.m7g.large \
    --allocated-storage 100 --storage-type gp3 --multi-az \
    --master-username app --manage-master-user-password \
    --db-subnet-group-name shop-db --vpc-security-group-ids $DB_SG \
    --no-publicly-accessible --storage-encrypted --backup-retention-period 14 \
    --enable-iam-database-authentication --deletion-protection

aws rds create-db-cluster --db-cluster-identifier shop-aurora \
    --engine aurora-postgresql --serverless-v2-scaling-configuration \
    MinCapacity=0.5,MaxCapacity=16 --master-username app \
    --manage-master-user-password --db-subnet-group-name shop-db
aws rds create-db-instance --db-instance-identifier shop-aurora-1 \
    --db-cluster-identifier shop-aurora --engine aurora-postgresql \
    --db-instance-class db.serverless

aws rds restore-db-instance-to-point-in-time \
    --source-db-instance-identifier shop-pg \
    --target-db-instance-identifier shop-pg-restore \
    --restore-time 2026-11-12T08:00:00Z
```

CODE EXPLAINED

- → Subnet groups place the database in private data subnets
- → Multi-AZ keeps a synchronous standby in another AZ
- → Aurora Serverless v2 scales between 0.5 and 16 ACUs
- → PITR restores into a new instance you can inspect

AZURE / GCP EQUIVALENT

↔ **Azure**	Azure SQL, PostgreSQL Flexible Server (az-31)
↔ **GCP**	Cloud SQL, AlloyDB (gcp-31)

COST CHECK

- $ Instance hours + storage + I/O (Aurora Standard) or Aurora I/O-Optimized
- $ Multi-AZ roughly doubles instance cost
- $ Reserved Instances discount steady databases; Graviton classes cost less

CHOOSING

OPTION	USE
RDS (single / Multi-AZ)	Standard managed DB
Multi-AZ DB cluster	Faster failover, readable standbys
Aurora provisioned	High throughput, fast replicas
Aurora Serverless v2	Variable load
Aurora DSQL	Distributed, multi-Region SQL
RDS Proxy	Connection pooling

RECOVERY

FEATURE	RPO / USE
Automated backups + PITR	Seconds–minutes
Manual snapshots	Kept until deleted
Read replica promotion	Regional DR
Aurora Global Database	Cross-Region, <1 s lag

DO & DON'T

- ✓ **Do** Turn on deletion protection and Multi-AZ for production
- ✓ **Do** Use RDS Proxy with Lambda
- ✗ **Don't** Expose databases publicly
- ✗ **Don't** Change parameter groups in production without a test

LOCAL TESTING

- ⌂ Run the same engine version in Docker
- ⌂ Testcontainers for integration tests
- ⌂ Restore a snapshot to a scratch instance for realistic testing

PRODUCTION CHECKLIST

- ☐ Multi-AZ, encryption, deletion protection
- ☐ Performance Insights / Database Insights and alarms
- ☐ Restore tested; backup retention per compliance

DynamoDB

SERVERLESS KEY-VALUE AND DOCUMENT DATABASE

DynamoDB stores items in tables keyed by a **partition key** (and optional **sort key**). Access is by key: `GetItem`, `Query` within a partition, or `Scan` (expensive). **Global secondary indexes** add alternate access patterns. Capacity is **on-demand** (pay per request) or **provisioned** with autoscaling. **TTL** expires items; **Streams** feed changes to Lambda.

KEY POINTS

- Design tables around access patterns first (single-table design is common)
- Strongly consistent reads cost 2× eventually consistent ones
- Global tables replicate across Regions (multi-active)
- PartiQL offers SQL-like syntax

SECURITY CHECK

- ⚠ Least-privilege IAM with `dynamodb:LeadingKeys` conditions for per-user data
- ⚠ Point-in-time recovery and deletion protection on
- ⚠ VPC gateway endpoint for private access

LIMITS & QUOTAS

- ≤ Item size up to 400 KB
- ≤ Per-partition throughput about 3,000 RCU / 1,000 WCU
- ≤ 20 global secondary indexes per table by default

TABLE, ITEMS, QUERIES

```
aws dynamodb create-table --table-name orders \
    --attribute-definitions AttributeName=pk,AttributeType=S \
      AttributeName=sk,AttributeType=S AttributeName=status,AttributeType=S \
    --key-schema AttributeName=pk,KeyType=HASH AttributeName=sk,KeyType=RANGE \
    --global-secondary-indexes file://gsi-by-status.json \
    --billing-mode PAY_PER_REQUEST --deletion-protection-enabled
aws dynamodb update-time-to-live --table-name orders \
    --time-to-live-specification Enabled=true,AttributeName=expires
aws dynamodb update-continuous-backups --table-name orders \
    --point-in-time-recovery-specification PointInTimeRecoveryEnabled=true
```

CODE EXPLAINED

- → Composite keys group a customer's orders in one partition
- → The GSI supports "orders by status"
- → TTL deletes items after the `expires` epoch time
- → `query` with `begins_with` reads one customer's month efficiently

PYTHON (BOTO3 RESOURCE)

```
import boto3
from boto3.dynamodb.conditions import Key
t = boto3.resource("dynamodb").Table("orders")
t.put_item(Item={"pk": "CUST#7", "sk": "ORDER#2026-11-12#42",
                 "status": "PAID", "total": 59})
t.get_item(Key={"pk": "CUST#7", "sk": "ORDER#2026-11-12#42"})
recent = t.query(KeyConditionExpression=Key("pk").eq("CUST#7")
                 & Key("sk").begins_with("ORDER#2026-11"))["Items"]
paid = t.query(IndexName="by-status",
               KeyConditionExpression=Key("status").eq("PAID"))["Items"]
```

AZURE / GCP EQUIVALENT

- ↔ **Azure** Cosmos DB: partition keys, RUs, consistency levels (az-32)
- ↔ **GCP** Firestore, Bigtable, Spanner (gcp-32)

COST CHECK

- $ On-demand: per million read / write request units; provisioned: per RCU / WCU-hour
- $ Storage per GB-month; GSIs store and bill their own copies
- $ Scans read (and bill) every item they touch

OPERATIONS

OPERATION	USE
`GetItem`	One item by full key
`Query`	Items in one partition, sort-key range
`Scan`	Whole table (avoid on hot paths)
`BatchGet / BatchWrite`	Up to 100 / 25 items
`TransactWriteItems`	All-or-nothing writes
`UpdateItem`	Conditional updates, counters

CAPACITY MODES

MODE	BEST FOR
`On-demand`	Unknown / spiky traffic
`Provisioned + autoscaling`	Steady, predictable traffic
`Reserved capacity`	Large steady provisioned use

DO & DON'T

- ✓ **Do** Model access patterns before creating tables
- ✓ **Do** Use condition expressions for safe concurrent updates
- ✗ **Don't** Scan in request paths
- ✗ **Don't** Use low-cardinality partition keys (hot partitions)

LOCAL TESTING

- ⌂ DynamoDB Local runs in Docker (`amazon/dynamodb-local`)
- ⌂ NoSQL Workbench models tables and generates code
- ⌂ moto mocks DynamoDB in tests

PRODUCTION CHECKLIST

- ☐ PITR, deletion protection, encryption with a customer managed key if required
- ☐ Alarms on throttled requests and errors
- ☐ Global tables if multi-Region active is needed

Sheet 33

ElastiCache & Other Databases

ElastiCache (Valkey / Redis OSS, Memcached) · DocumentDB · OpenSearch · Neptune · choosing a database · pointer to SQL & NoSQL sets

CACHES AND PURPOSE-BUILT DATABASES

ElastiCache runs Valkey (recommended, lower cost), Redis OSS or Memcached, as node-based clusters or **Serverless**. **DocumentDB** is MongoDB-compatible, **OpenSearch Service** handles search and log analytics (including vector search), **Neptune** is a graph database, **Timestream** stores time series, and **MemoryDB** is a durable Valkey / Redis-compatible primary database.

KEY POINTS

- Cache-aside: read cache, fall back to the database, then populate
- ElastiCache Serverless scales automatically with no node sizing
- OpenSearch Serverless removes cluster management
- Keep caches in private subnets with TLS and auth

VALKEY SERVERLESS CACHE

```
aws elasticache create-serverless-cache --serverless-cache-name shop-cache \
    --engine valkey --subnet-ids $DATA_A $DATA_B \
    --security-group-ids $CACHE_SG
aws elasticache describe-serverless-caches \
    --query "ServerlessCaches[].Endpoint" --output table

aws opensearchserverless create-collection --name shop-search --type SEARCH
```

CODE EXPLAINED

- → Serverless caches need only subnets and security groups
- → Valkey is wire-compatible with Redis clients
- → An OpenSearch Serverless collection still needs encryption / network / access policies
- → Entries expire after 300 seconds

CACHE-ASIDE IN PYTHON

```
import json, redis
r = redis.Redis(host="shop-cache-abc.serverless.use1.cache.amazonaws.com",
                port=6379, ssl=True)

def get_product(pid):
    if (hit := r.get(f"product:{pid}")):
        return json.loads(hit)
    row = db_fetch_product(pid)                 # slow path
    r.set(f"product:{pid}", json.dumps(row), ex=300)
    return row
```

AZURE / GCP EQUIVALENT

- ↔ **Azure** Azure Managed Redis, AI Search (az-33)
- ↔ **GCP** Memorystore, BigQuery, Vertex AI Search (gcp-33)

COST CHECK

- $ Valkey pricing is lower than Redis OSS on ElastiCache
- $ Serverless caches bill per GB stored + processing units
- $ OpenSearch bills instances / OCUs plus storage

CHOOSING A DATABASE

NEED	USE
Relational	RDS / Aurora (sheet 31)
Key-value at scale	DynamoDB (sheet 32)
Cache	ElastiCache (Valkey)
Durable in-memory	MemoryDB
Document (MongoDB API)	DocumentDB
Search / logs / vectors	OpenSearch
Graph	Neptune
Time series	Timestream
Analytics	Redshift / Athena

DO & DON'T

- ✓ **Do** Set TTLs on every cache key
- ✓ **Do** Choose Valkey for new caches
- ✗ **Don't** Treat a cache as the source of truth
- ✗ **Don't** Pick a niche database before checking if PostgreSQL fits

LOCAL TESTING

- ⌂ Run Valkey / Redis in Docker: `docker run -p 6379:6379 valkey/valkey`
- ⌂ OpenSearch and MongoDB containers for local search / document tests
- ⌂ Mock the cache layer in unit tests

PRODUCTION CHECKLIST

- ☐ Encryption in transit + at rest, auth enabled
- ☐ Multi-AZ / replicas for caches that matter
- ☐ Hit-rate, memory and eviction alarms

CloudFormation

template anatomy (`Parameters`, `Resources`, `Outputs`) · `!Ref` / `!Sub` / `!GetAtt` · stacks · change sets · drift detection · `aws cloudformation deploy`

AWS'S NATIVE IAC

CloudFormation creates and updates **stacks** of resources from YAML or JSON **templates**. Templates declare `Parameters`, `Resources` and `Outputs`, with intrinsic functions like `!Ref`, `!Sub` and `!GetAtt`. **Change sets** preview updates, **drift detection** finds manual changes, and **StackSets** deploy across accounts and Regions. Failed updates roll back automatically.

KEY POINTS

- `aws cloudformation deploy` creates or updates a stack via a change set
- `DeletionPolicy: Retain` / `Snapshot` protects data on delete
- Stack policies and termination protection guard critical stacks
- cfn-lint and cfn-guard validate templates

TEMPLATE.YAML

```yaml
AWSTemplateFormatVersion: "2010-09-09"
Parameters:
  Env: { Type: String, AllowedValues: [dev, prod], Default: dev }
Conditions:
  IsProd: !Equals [!Ref Env, prod]
Resources:
  Uploads:
    Type: AWS::S3::Bucket
    DeletionPolicy: Retain
    Properties:
      BucketName: !Sub "acme-uploads-${Env}-${AWS::AccountId}"
      VersioningConfiguration:
        Status: !If [IsProd, Enabled, Suspended]
      PublicAccessBlockConfiguration:
        { BlockPublicAcls: true, BlockPublicPolicy: true,
          IgnorePublicAcls: true, RestrictPublicBuckets: true }
Outputs:
  BucketArn: { Value: !GetAtt Uploads.Arn }
  BucketName: { Value: !Ref Uploads, Export: { Name: !Sub "${Env}-uploads" } }
```

CODE EXPLAINED

- → `!Sub` builds names from parameters and pseudo parameters
- → The condition enables versioning only in prod
- → `Retain` keeps the bucket if the stack is deleted
- → `--no-execute-changeset` previews before applying

AZURE / GCP EQUIVALENT

- ↔ **Azure** Bicep, what-if, deployment stacks (az-34)
- ↔ **GCP** Terraform, Infrastructure Manager (gcp-34, gcp-35)

DEPLOY, PREVIEW, DRIFT

```
aws cloudformation validate-template --template-body file://template.yaml
aws cloudformation deploy --stack-name shop-prod --template-file template.yaml \
    --parameter-overrides Env=prod --no-execute-changeset
aws cloudformation deploy --stack-name shop-prod --template-file template.yaml \
    --parameter-overrides Env=prod
aws cloudformation detect-stack-drift --stack-name shop-prod
```

COST CHECK

- $ CloudFormation is free for AWS resource types
- $ Third-party / private resource types bill per handler operation
- $ Change sets catch expensive resource replacements before they happen

SECURITY CHECK

- ⚠ Never put secrets in parameters defaults or outputs; use dynamic references (`{{resolve:secretsmanager:…}}`)
- ⚠ Termination protection and stack policies on production stacks
- ⚠ Deploy roles (`--role-arn`) instead of broad user permissions

LIMITS & QUOTAS

- ≤ 500 resources per stack
- ≤ 200 parameters and 200 outputs per template
- ≤ Template body up to 51,200 bytes inline; 1 MB from S3

INTRINSIC FUNCTIONS

FUNCTION	DOES
`!Ref`	Parameter value / resource ID
`!GetAtt R.Attr`	Resource attribute
`!Sub`	String interpolation
`!If / !Equals`	Conditions
`!ImportValue`	Cross-stack output
`!Join / !Select / !Split`	List helpers

FEATURES

FEATURE	USE
Change sets	Preview updates
Drift detection	Find manual changes
StackSets	Multi-account / Region
Nested stacks / modules	Reuse
Hooks / Guard	Policy checks

DO & DON'T

- ✓ **Do** Preview with change sets before production updates
- ✓ **Do** Use DeletionPolicy on data resources
- ✗ **Don't** Edit stack resources by hand in the console
- ✗ **Don't** Hard-code account IDs and Regions (use pseudo parameters)

LOCAL TESTING

- ⌂ cfn-lint and cfn-guard run offline
- ⌂ LocalStack deploys many CloudFormation resources locally
- ⌂ Rain CLI formats and deploys templates with nicer output

PRODUCTION CHECKLIST

- ☐ Deploy from CI with a dedicated deploy role and OIDC
- ☐ Termination protection + drift detection schedule
- ☐ Templates linted and policy-checked in pull requests

AWS CDK

`cdk init` · constructs (L1 / L2 / L3) · stacks & apps · `cdk bootstrap` · `cdk synth` / `diff` / `deploy` · TypeScript & Python

INFRASTRUCTURE IN A PROGRAMMING LANGUAGE

The **AWS Cloud Development Kit (CDK)** defines infrastructure in TypeScript, Python, Java, C# or Go, then **synthesizes** CloudFormation templates. **Constructs** come in levels: L1 (raw CloudFormation), L2 (sensible defaults and helper methods like `grantRead`) and L3 patterns (whole architectures). An **app** contains **stacks**; `cdk deploy` ships them.

KEY POINTS

- `cdk bootstrap` creates the staging bucket and roles once per account / Region
- L2 `grant*` methods write least-privilege IAM for you
- cdk-nag checks constructs against best-practice rules
- Construct Hub lists community libraries

LIB/SHOP-STACK.TS

```ts
import * as cdk from "aws-cdk-lib";
import { Construct } from "constructs";
import * as s3 from "aws-cdk-lib/aws-s3";
import * as lambda from "aws-cdk-lib/aws-lambda";
import * as sqs from "aws-cdk-lib/aws-sqs";
import { SqsEventSource } from "aws-cdk-lib/aws-lambda-event-sources";

export class ShopStack extends cdk.Stack {
  constructor(scope: Construct, id: string, props?: cdk.StackProps) {
    super(scope, id, props);
    const bucket = new s3.Bucket(this, "Uploads", {
      versioned: true, enforceSSL: true,
      blockPublicAccess: s3.BlockPublicAccess.BLOCK_ALL,
      removalPolicy: cdk.RemovalPolicy.RETAIN,
    });
    const queue = new sqs.Queue(this, "Orders", {
      visibilityTimeout: cdk.Duration.seconds(60) });
    const fn = new lambda.Function(this, "Worker", {
      runtime: lambda.Runtime.NODEJS_22_X,
      architecture: lambda.Architecture.ARM_64,
      handler: "index.handler", code: lambda.Code.fromAsset("lambda"),
      environment: { BUCKET: bucket.bucketName } });
    bucket.grantReadWrite(fn);                // least-privilege IAM
    fn.addEventSource(new SqsEventSource(queue, { batchSize: 10 }));
  }
}
```

CODE EXPLAINED

- → L2 constructs set secure defaults like `enforceSSL`
- → `grantReadWrite` adds exactly the S3 permissions the function needs
- → `addEventSource` wires the SQS trigger and its IAM
- → `diff` shows template and IAM changes before deploy

AZURE / GCP EQUIVALENT

↔	**Azure**	Bicep modules / Azure Developer CLI (az-34, az-35)
↔	**GCP**	Terraform modules / CDK for Terraform (gcp-34)

CLI

```
npx cdk init app --language typescript
npx cdk bootstrap aws://111122223333/us-east-1
npx cdk synth && npx cdk diff && npx cdk deploy --require-approval broadening
```

COST CHECK

- $ CDK is free; it deploys through CloudFormation
- $ Bootstrap creates a small S3 bucket and ECR repo for assets
- $ cdk diff reveals costly replacements (e.g. renaming a database)

CONSTRUCT LEVELS

LEVEL	EXAMPLE
`L1 (Cfn*)`	CfnBucket: 1:1 with CloudFormation
`L2`	s3.Bucket with defaults + grants
`L3 / patterns`	ApplicationLoadBalancedFargateService

COMMANDS

COMMAND	DOES
`cdk init`	New project
`cdk bootstrap`	One-time per env
`cdk synth`	Template out
`cdk diff`	Preview
`cdk deploy / destroy`	Apply / remove
`cdk watch`	Fast dev deploys

DO & DON'T

- ✓ **Do** Use L2 constructs and `grant*` methods
- ✓ **Do** Run cdk-nag in CI
- ✗ **Don't** Rename construct IDs of stateful resources (causes replacement)
- ✗ **Don't** Put environment-specific logic everywhere instead of stack props

LOCAL TESTING

- ⌂ `cdk synth` runs fully offline
- ⌂ Jest / pytest assertions test synthesized templates
- ⌂ LocalStack `cdklocal` deploys to the local emulator

PRODUCTION CHECKLIST

- ☐ Pipelines (CDK Pipelines or GitHub Actions) deploy, not laptops
- ☐ RemovalPolicy.RETAIN on data resources
- ☐ cdk-nag and diff review in pull requests

Sheet 36

SAM & Terraform on AWS

`sam init` / `build` / `deploy` · `AWS::Serverless` transform · `sam local invoke` · Terraform AWS provider · S3 state backend · choosing an IaC tool

SERVERLESS TEMPLATES AND TERRAFORM

The **AWS Serverless Application Model (SAM)** extends CloudFormation with short resource types (`AWS::Serverless::Function`, `Api`, `SimpleTable`) and a CLI that builds, runs locally and deploys. **Terraform** with the AWS provider is the common multi-cloud choice; state goes in **S3** with native locking (lock files).

KEY POINTS

- `sam deploy --guided` saves settings in `samconfig.toml`
- `sam sync --watch` pushes code changes in seconds during development
- Terraform S3 backend supports `use_lockfile = true` (DynamoDB locking is legacy)
- Use OIDC roles for Terraform in CI

TEMPLATE.YAML (SAM)

```yaml
Transform: AWS::Serverless-2016-10-31
Globals:
  Function: { Runtime: python3.13, Architectures: [arm64], Timeout: 10 }
Resources:
  OrdersFn:
    Type: AWS::Serverless::Function
    Properties:
      Handler: app.handler
      CodeUri: src/
      Policies: [{ DynamoDBCrudPolicy: { TableName: !Ref Orders } }]
      Events:
        Api: { Type: HttpApi, Properties: { Path: /orders, Method: post } }
  Orders:
    Type: AWS::Serverless::SimpleTable
```

CODE EXPLAINED

- → The SAM transform expands short types into full CloudFormation
- → Policy templates like `DynamoDBCrudPolicy` scope IAM to one table
- → `default_tags` tags every AWS resource Terraform creates
- → The S3 backend locks state with a lock file

TERRAFORM

```hcl
terraform {
  required_providers { aws = { source = "hashicorp/aws", version = "~> 6.0" } }
  backend "s3" {
    bucket       = "acme-tfstate"
    key          = "shop/prod.tfstate"
    region       = "us-east-1"
    use_lockfile = true
  }
}
provider "aws" {
  region = "us-east-1"
  default_tags { tags = { owner = "team-a", env = "prod" } }
}
resource "aws_sqs_queue" "orders" { visibility_timeout_seconds = 60 }
```

AZURE / GCP EQUIVALENT

- ↔ **Azure** Azure Developer CLI, Terraform azurerm (az-35, az-36)
- ↔ **GCP** Terraform, Infrastructure Manager, Config Connector (gcp-34–36)

CLIS

```
sam build && sam local invoke OrdersFn -e event.json && sam deploy --guided
terraform init && terraform plan -out tf.plan && terraform apply tf.plan
```

COST CHECK

- $ SAM and Terraform CLI are free
- $ State bucket costs pennies; enable versioning on it
- $ Infracost estimates Terraform plan costs in PRs

CHOOSING IAC ON AWS

TOOL	BEST WHEN
CloudFormation	AWS-only, declarative YAML
SAM	Serverless apps with local testing
CDK	Teams who prefer code, rich constructs
Terraform / OpenTofu	Multi-cloud, existing Terraform skills
Pulumi	Code with multi-cloud state

DO & DON'T

- ✓ **Do** Keep one state file per environment / component
- ✓ **Do** Use SAM policy templates instead of broad managed policies
- ✗ **Don't** Store Terraform state locally or in git
- ✗ **Don't** Mix console edits with IaC-managed resources

LOCAL TESTING

- ⌂ `sam local start-api` / `invoke` run functions in Docker
- ⌂ `terraform validate` and tflint run offline
- ⌂ LocalStack works with both SAM (`samlocal`) and Terraform (`tflocal`)

PRODUCTION CHECKLIST

- ☐ CI deploys with OIDC roles and approvals
- ☐ State bucket versioned, encrypted, private
- ☐ Drift detection (terraform plan on a schedule)

CloudWatch Metrics & Alarms

WATCHING AWS RESOURCES

CloudWatch collects metrics from AWS services in **namespaces** (AWS/EC2, AWS/Lambda) with **dimensions** (InstanceId, FunctionName). Publish your own with put-metric-data or embedded metric format (EMF) logs. **Alarms** watch a metric or expression and notify SNS, scale ASGs or run actions. **Composite alarms** combine alarms; **anomaly detection** learns normal bands.

KEY POINTS

- Basic monitoring: 5-minute EC2 metrics; detailed: 1-minute
- Metric math builds error rates from counts
- Alarms have OK / ALARM / INSUFFICIENT_DATA states
- Application Signals adds SLOs and service maps

CUSTOM METRIC, ALARM, DASHBOARD

```
aws cloudwatch put-metric-data --namespace Shop \
    --metric-name OrdersPlaced --dimensions Env=prod --value 1 --unit Count

aws sns create-topic --name oncall
aws cloudwatch put-metric-alarm --alarm-name api-5xx \
    --namespace AWS/ApplicationELB --metric-name HTTPCode_Target_5XX_Count \
    --dimensions Name=LoadBalancer,Value=app/shop-alb/abc123 \
    --statistic Sum --period 60 --evaluation-periods 5 --datapoints-to-alarm 3 \
    --threshold 10 --comparison-operator GreaterThanThreshold \
    --treat-missing-data notBreaching --alarm-actions $ONCALL_TOPIC

aws cloudwatch put-metric-alarm --alarm-name lambda-errors \
    --namespace AWS/Lambda --metric-name Errors \
    --dimensions Name=FunctionName,Value=orders --statistic Sum \
    --period 300 --evaluation-periods 1 --threshold 1 \
    --comparison-operator GreaterThanOrEqualToThreshold \
    --alarm-actions $ONCALL_TOPIC

aws cloudwatch get-metric-statistics --namespace AWS/Lambda \
    --metric-name Duration --dimensions Name=FunctionName,Value=orders \
    --start-time 2026-11-12T00:00:00Z --end-time 2026-11-12T12:00:00Z \
    --period 3600 --statistics Average Maximum
aws cloudwatch put-dashboard --dashboard-name shop \
    --dashboard-body file://dash.json
```

CODE EXPLAINED

- → Custom metrics live in your own namespace
- → 3 of 5 one-minute periods over 10 errors trigger the alarm
- → Missing data is treated as OK for sparse metrics
- → Dashboards are JSON documents you can keep in IaC

AZURE / GCP EQUIVALENT

- ↔ **Azure** Azure Monitor metrics, alert rules, action groups (az-37)
- ↔ **GCP** Cloud Monitoring, alerting policies (gcp-37)

COST CHECK

- $ Basic AWS service metrics are free; custom metrics bill per metric per month
- $ Alarms bill per alarm metric; high-resolution costs more
- $ High-cardinality dimensions multiply custom metric costs

KEY METRICS

SERVICE	WATCH
ALB	HTTPCode_Target_5XX_Count, TargetResponseTime
Lambda	Errors, Throttles, Duration
SQS	ApproximateAgeOfOldestMessage
RDS	CPUUtilization, FreeStorageSpace
DynamoDB	ThrottledRequests
EC2	StatusCheckFailed

ALARM ACTIONS

ACTION	DOES
SNS	Notify people / systems
Auto Scaling	Scale a group
EC2 action	Stop / reboot / recover
Systems Manager	OpsItem / incident
Lambda (via SNS / EventBridge)	Automation

DO & DON'T

- ✓ **Do** Alarm on user-facing symptoms (errors, latency, queue age)
- ✓ **Do** Keep alarms and dashboards in IaC
- ✗ **Don't** Create custom metrics with user IDs as dimensions
- ✗ **Don't** Page on single noisy data points

LOCAL TESTING

- ⌂ Emit EMF logs locally and inspect the JSON
- ⌂ LocalStack emulates CloudWatch APIs for tests
- ⌂ Test alarm thresholds against historical data with `get-metric-statistics`

PRODUCTION CHECKLIST

- ☐ Alarms with runbooks routed to on-call
- ☐ Composite alarms to cut noise
- ☐ Dashboards per service with golden signals

CloudWatch Logs & Tracing

log groups & streams · retention · Logs Insights queries · metric filters · X-Ray / OpenTelemetry (ADOT) · `aws logs tail --follow`

LOGS, QUERIES AND TRACES

CloudWatch Logs stores events in **log groups** (one per app / function) and **streams**. Set **retention** on every group: the default is to keep logs forever. **Logs Insights** queries logs with a pipe-based language; **metric filters** turn patterns into metrics. Traces flow through **X-Ray** or the **AWS Distro for OpenTelemetry (ADOT)** into CloudWatch Application Signals.

KEY POINTS

- Lambda, ECS (awslogs) and API Gateway write to log groups automatically
- Structured JSON logs make fields queryable
- Subscription filters stream logs to Lambda, Firehose or OpenSearch
- Infrequent Access log class cuts ingestion cost for rarely queried logs

RETENTION, TAIL, QUERY, METRIC FILTER

```
aws logs put-retention-policy --log-group-name /aws/lambda/orders \
    --retention-in-days 30
aws logs tail /aws/lambda/orders --follow --since 10m --format short
aws logs put-metric-filter --log-group-name /ecs/shop-api \
    --filter-name errors --filter-pattern '{ $.level = "error" }' \
    --metric-transformations \
    metricName=ApiErrors,metricNamespace=Shop,metricValue=1
aws logs start-query --log-group-names /ecs/shop-api \
    --start-time $(date -d '-1 hour' +%s) --end-time $(date +%s) \
    --query-string file://errors.query
```

CODE EXPLAINED

→ 30-day retention stops logs from accumulating forever

→ `tail --follow` streams new events like `tail -f`

→ The JSON metric filter counts error-level log lines

→ The query finds slow or failing paths with p95 latency

LOGS INSIGHTS QUERY

```
fields @timestamp, level, path, duration_ms
| filter level = "error" or duration_ms > 1000
| stats count() as n, pct(duration_ms, 95) as p95 by path
| sort p95 desc
| limit 20
```

AZURE / GCP EQUIVALENT

↔ **Azure** Log Analytics (KQL), Application Insights (az-38)

↔ **GCP** Cloud Logging, Cloud Trace (gcp-38)

COST CHECK

$ Ingestion per GB is the main cost; Infrequent Access class is cheaper

$ Storage per GB-month after ingestion

$ Logs Insights bills per GB scanned: narrow time ranges and groups

TRACING WITH OPENTELEMETRY (PYTHON)

```
from opentelemetry import trace
tracer = trace.get_tracer("shop")
with tracer.start_as_current_span("checkout") as span:
    span.set_attribute("cart.items", 3)     # exported via ADOT / CW agent
```

LIMITS & QUOTAS

≤ Default retention: never expire (set it)

≤ Logs Insights queries time out after 60 minutes

≤ PutLogEvents batches up to 1 MB / 10,000 events

INSIGHTS COMMANDS

COMMAND	DOES
`fields`	Pick fields
`filter`	Rows matching
`stats … by`	Aggregate
`sort / limit`	Order and cap
`parse`	Extract from text
`pattern`	Cluster similar logs

TRACING OPTIONS

OPTION	USE
`ADOT / OpenTelemetry`	Vendor-neutral, recommended
`X-Ray SDK`	Legacy instrumentation
`Application Signals`	SLOs, service map
`Lambda active tracing`	One flag per function

DO & DON'T

✓ **Do** Set retention on every log group (enforce with Config / IaC)

✓ **Do** Log JSON with request and trace IDs

✗ **Don't** Log secrets or full request bodies with PII

✗ **Don't** Query 30 days of all log groups for a one-hour incident

LOCAL TESTING

⌂ OpenTelemetry Collector + Jaeger in Docker for local traces

⌂ `sam local` prints Lambda logs to the terminal

⌂ Test Insights queries on a small time window first

PRODUCTION CHECKLIST

☐ Retention, log class and encryption set per group

☐ Metric filters / alarms for error patterns

☐ Tracing across services with ADOT

Deploying to AWS

CI/CD INTO AWS

Pipelines should assume an IAM role through **OIDC**: GitHub Actions uses `aws-actions/configure-aws-credentials` with a role whose trust policy pins the repo and environment. Build images and push to ECR, then deploy with CloudFormation / CDK / SAM / Terraform or service commands. **CodeDeploy** runs blue/green and canary rollouts for ECS, Lambda and EC2.

KEY POINTS

- No access keys in CI secrets
- Separate deploy roles per environment and account
- CodePipeline / CodeBuild are AWS-native alternatives to GitHub Actions
- Tag resources with repo, commit and owner

GITHUB ACTIONS: BUILD, PUSH, DEPLOY TO ECS

```yaml
on: { push: { branches: [main] } }
permissions: { id-token: write, contents: read }
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::111122223333:role/gh-deploy-prod
          aws-region: us-east-1
      - uses: aws-actions/amazon-ecr-login@v2
        id: ecr
      - run: |
          IMG=${{ steps.ecr.outputs.registry }}/shop-api:${{ github.sha }}
          docker build -t $IMG . && docker push $IMG
          echo "IMG=$IMG" >> $GITHUB_ENV
      - run: |
          aws ecs describe-task-definition --task-definition shop-api \
            --query taskDefinition > td.json
      - uses: aws-actions/amazon-ecs-render-task-definition@v1
        id: td
        with:
          task-definition: td.json
          container-name: api
          image: ${{ env.IMG }}
      - uses: aws-actions/amazon-ecs-deploy-task-definition@v2
        with: { task-definition: "${{ steps.td.outputs.task-definition }}",
                service: api, cluster: shop, wait-for-service-stability: true }
```

CODE EXPLAINED

- → `id-token: write` lets the job request the OIDC token
- → The role is assumed with no stored secret
- → The image tag is the commit SHA for traceability
- → Render + deploy updates the ECS service and waits for stability

AZURE / GCP EQUIVALENT

- ↔ **Azure** azure/login OIDC, Azure Pipelines, slot swaps (az-39)
- ↔ **GCP** google-github-actions/auth WIF, Cloud Build, Cloud Deploy (gcp-39)

COST CHECK

- $ GitHub-hosted runner minutes / CodeBuild per build-minute
- $ CodeDeploy is free for ECS / Lambda / EC2 (on-premises costs extra)
- $ Blue/green briefly runs double capacity

SECURITY CHECK

- ⚠ Trust policy `sub` pinned to `repo:org/app:environment:production`
- ⚠ Deploy role scoped to the services and resources it changes
- ⚠ Pin third-party actions to a commit SHA

DEPLOYMENT STRATEGIES

TARGET	STRATEGY
ECS	Rolling, blue/green (CodeDeploy or native)
Lambda	Alias canary / linear via CodeDeploy
EC2 / ASG	Instance refresh, CodeDeploy
Infra	CloudFormation change sets / Terraform plans
Static sites	S3 sync + CloudFront invalidation

PIPELINE PIECES

PIECE	AWS NATIVE	COMMON ALTERNATIVE
Source	CodeConnections	GitHub
Build	CodeBuild	GitHub Actions
Deploy	CodeDeploy / CloudFormation	Actions + CLI
Orchestrate	CodePipeline	Workflow YAML

DO & DON'T

- ✓ **Do** Require environment approvals for production
- ✓ **Do** Wait for service stability and fail the job on rollback
- ✗ **Don't** Store AWS keys in GitHub secrets
- ✗ **Don't** Use the same role for every environment

LOCAL TESTING

- ⌂ `act` runs workflows locally (OIDC needs real GitHub)
- ⌂ actionlint validates workflow files
- ⌂ Run the same CLI steps from a laptop against dev first

PRODUCTION CHECKLIST

- ☐ OIDC role per environment with least privilege
- ☐ Approval gates and deployment alarms with auto-rollback
- ☐ Every deploy traceable to a commit (tags, image SHA)

Amazon Bedrock

model access · `InvokeModel` vs Converse API · streaming · Knowledge Bases (RAG) · Agents · Guardrails

GENERATIVE AI ON AWS

Amazon Bedrock offers foundation models from Anthropic, Amazon (Nova), Meta, Mistral, Cohere and others behind one API. The **Converse API** gives a uniform chat interface with tools and streaming across models; `InvokeModel` sends model-specific payloads. **Cross-Region inference profiles** spread load. **Knowledge Bases** add managed RAG, **Agents** add tool use, and **Guardrails** filter content and PII.

CONVERSE API (PYTHON)

```python
import boto3
brt = boto3.client("bedrock-runtime", region_name="us-east-1")
MODEL = "us.amazon.nova-lite-v1:0"        # inference profile ID (example)

resp = brt.converse(
    modelId=MODEL,
    system=[{"text": "You are a concise support assistant."}],
    messages=[{"role": "user", "content": [
        {"text": "Summarize our return policy in 3 bullets."}]}],
    inferenceConfig={"maxTokens": 300, "temperature": 0.2})
print(resp["output"]["message"]["content"][0]["text"])
print(resp["usage"])                      # input / output tokens

stream = brt.converse_stream(modelId=MODEL, messages=[{"role": "user",
    "content": [{"text": "Write a haiku about queues."}]}])
for ev in stream["stream"]:
    if "contentBlockDelta" in ev:
        print(ev["contentBlockDelta"]["delta"]["text"], end="")
```

CLI

```
aws bedrock list-foundation-models --by-output-modality TEXT \
    --query "modelSummaries[].modelId" --output text
aws bedrock list-inference-profiles \
    --query "inferenceProfileSummaries[].inferenceProfileId"
aws bedrock create-guardrail --name support --blocked-input-messaging "Sorry." \
    --blocked-outputs-messaging "Sorry." --cli-input-json file://guardrail.json
```

KEY POINTS

- Serverless models are enabled automatically on first use (some providers need a one-time use-case form)
- Model IDs or inference profile IDs select the model
- Data is not used to train the models
- Batch inference costs less for offline jobs

CODE EXPLAINED

- → Converse works the same across model providers
- → Inference profile IDs (`us.` prefix) route across US Regions
- → `usage` reports tokens for cost tracking
- → Streaming prints text as it arrives

AZURE / GCP EQUIVALENT

- ↔ **Azure** Microsoft Foundry / Azure OpenAI (az-40)
- ↔ **GCP** Agent Platform (Vertex AI) + Gemini (gcp-40)

COST CHECK

- $ On-demand: per input / output token, varies by model
- $ Batch inference is about half the on-demand price for supported models
- $ Smaller models (Nova Micro / Lite, Haiku-class) cost far less for simple tasks

SECURITY CHECK

- ⚠ IAM policies scoped to specific model / inference profile ARNs
- ⚠ VPC interface endpoint (bedrock-runtime) for private access
- ⚠ Guardrails for PII and harmful content on user-facing apps

LIMITS & QUOTAS

- ≤ Quotas per model per Region: tokens per minute and requests per minute
- ≤ Throttling returns ThrottlingException: retry with backoff
- ≤ Provisioned Throughput reserves capacity for steady high volume

BUILDING BLOCKS

FEATURE	DOES
`Converse / ConverseStream`	Unified chat + tools
`InvokeModel`	Model-native payloads
`Knowledge Bases`	Managed RAG over S3 and more
`Agents / AgentCore`	Tool-using agents and runtime
`Guardrails`	Content, PII, topic filters
`Batch inference`	Async jobs
`Model evaluation`	Compare models / prompts

THROUGHPUT OPTIONS

OPTION	USE
`On-demand`	Default pay per token
`Cross-Region inference`	Higher throughput across Regions
`Provisioned Throughput`	Reserved model units
`Batch`	Offline, cheaper

DO & DON'T

- ✓ **Do** Use the Converse API to keep model switching cheap
- ✓ **Do** Log token usage per feature for cost tracking
- ✗ **Don't** Send sensitive data without Guardrails / redaction where required
- ✗ **Don't** Retry throttles instantly in a tight loop

LOCAL TESTING

- ⌂ Prototype prompts in the Bedrock console playground
- ⌂ Mock `bedrock-runtime` responses in unit tests (botocore Stubber)
- ⌂ Run evaluation sets locally against a dev account

PRODUCTION CHECKLIST

- □ Least-privilege model access and VPC endpoints
- □ Guardrails and logging (model invocation logging) on
- □ Quota monitoring and backoff for throttling

AI & ML Services

PREBUILT AI AND CUSTOM ML

AWS offers task-specific AI APIs: **Rekognition** (images and video), **Textract** (forms, tables, documents), **Transcribe** (speech-to-text), **Polly** (text-to-speech), **Comprehend** (entities, sentiment, PII) and **Translate**. **SageMaker AI** trains, tunes and hosts custom models. **Amazon Q Developer** is the coding assistant.

KEY POINTS

- Prebuilt APIs need no training data
- Async APIs (Textract, Transcribe) work on files in S3
- SageMaker endpoints bill while running: delete idle ones
- Comprehend detects and redacts PII

TEXTRACT, COMPREHEND, TRANSCRIBE

```
import boto3
tx = boto3.client("textract")
doc = tx.analyze_expense(Document={"S3Object": {"Bucket": "acme-docs",
                                                "Name": "invoice.pdf"}})
for f in doc["ExpenseDocuments"][0]["SummaryFields"]:
    print(f["Type"]["Text"], "=", f.get("ValueDetection", {}).get("Text"))

cp = boto3.client("comprehend")
pii = cp.detect_pii_entities(Text="Call Ada at 555-0100 about order 42.",
                             LanguageCode="en")
print([e["Type"] for e in pii["Entities"]])   # ['NAME', 'PHONE']

ts = boto3.client("transcribe")
ts.start_transcription_job(TranscriptionJobName="call-42",
    Media={"MediaFileUri": "s3://acme-audio/call-42.mp3"},
    LanguageCode="en-US", OutputBucketName="acme-transcripts")
```

REKOGNITION AND SAGEMAKER

```
aws rekognition detect-labels \
    --image 'S3Object={Bucket=acme-media,Name=cat.jpg}' \
    --max-labels 5 --query "Labels[].[Name,Confidence]"
aws sagemaker list-endpoints --query "Endpoints[].[EndpointName,EndpointStatus]"
```

CODE EXPLAINED

- → `analyze_expense` extracts invoice totals and vendor fields
- → Comprehend returns PII entity types and offsets
- → Transcribe runs as a job and writes results to S3
- → Rekognition labels images straight from S3

AZURE / GCP EQUIVALENT

- ↔ **Azure** Foundry Tools (Vision, Speech, Language, Document Intelligence), Azure ML (az-41)
- ↔ **GCP** Vision API, Document AI, Speech-to-Text, Vertex AI training (gcp-41)

COST CHECK

- $ Per image / page / minute / character, most with free tiers for the first year
- $ SageMaker: instance hours for training, endpoints and notebooks
- $ Serverless and async inference avoid paying for idle endpoints

SERVICES

SERVICE	DOES
Rekognition	Labels, faces, text, moderation
Textract	Text, forms, tables, expenses, IDs
Transcribe	Speech-to-text, call analytics
Polly	Text-to-speech
Comprehend	Entities, sentiment, PII
Translate	Machine translation
SageMaker AI	Custom ML lifecycle
Q Developer	Coding assistant

DO & DON'T

- ✓ **Do** Try prebuilt APIs before training custom models
- ✓ **Do** Use async / batch APIs for large volumes
- ✗ **Don't** Leave SageMaker endpoints and notebooks running idle
- ✗ **Don't** Process personal data without checking retention and opt-out settings

LOCAL TESTING

- ⌂ Mock clients with botocore Stubber in tests
- ⌂ Test with small sample files in a dev account
- ⌂ SageMaker local mode runs training containers on your machine

PRODUCTION CHECKLIST

- ☐ IAM scoped per service and bucket
- ☐ Retries for throttling; quotas monitored
- ☐ Idle SageMaker resources cleaned up automatically

Sheet 42

Quick Reference — CLI & Service Picker

aws s3 · ec2 · iam / sts · lambda · ecr / ecs / eks · logs · cloudformation · Need → AWS service → Azure → GCP (with sheet numbers) · default limits worth knowing · pricing model per service

IDENTITY & ACCOUNT

COMMAND	DOES	SHEET #
aws sso login --profile p	Sign in	03
aws sts get-caller-identity	Who am I	03
aws iam create-role / attach-role-policy	Roles	05–06
aws sts assume-role	Temporary creds	07
aws secretsmanager get-secret-value	Read secret	08
aws budgets create-budget	Budget alert	02

COMPUTE

COMMAND	DOES	SHEET #
aws ec2 run-instances	Launch	15
aws ssm start-session --target i-…	Shell, no SSH	15
aws autoscaling create-auto-scaling-group	ASG	18
aws lambda create-function / invoke	Lambda	20
aws lambda update-alias	Canary	21
aws logs tail G --follow	Stream logs	38

CONTAINERS

COMMAND	DOES	SHEET #
aws ecr get-login-password | docker login	Registry login	25
aws ecs create-service / update-service	ECS	26
aws ecs execute-command	Shell into task	26
aws eks update-kubeconfig --name c	kubectl access	27
eksctl create cluster	EKS clusterer	27

STORAGE & DATA

COMMAND	DOES	SHEET #
aws s3 cp / sync / ls / rm	Objects	28
aws s3 presign	Temporary URL	29
aws rds create-db-instance	Database	31
aws dynamodb query / put-item	NoSQL	32
aws sqs send-message / receive-message	Queues	24

IAC & OPS

COMMAND	DOES	SHEET #
aws cloudformation deploy	Stack	34
npx cdk diff / deploy	CDK	35
sam build / local invoke / deploy	SAM	36
aws cloudwatch put-metric-alarm	Alarm	37
aws logs start-query	Insights	38

SERVICE PICKER

NEED	AWS	AZURE	GOOGLE CLOUD	SHEET #
People sign-in	IAM Identity Center	Entra ID	Cloud Identity	05
Workload identity	IAM role	Managed identity	Service account	07
Secrets	Secrets Manager	Key Vault	Secret Manager	08
Private network	VPC	VNet	VPC (global)	10
CDN	CloudFront	Front Door	Cloud CDN	14
VMs / scaling	EC2 / ASG	VMs / VMSS	Compute Engine / MIG	15, 18
Functions	Lambda	Azure Functions	Cloud Run functions	20
Containers (serverless)	ECS Fargate / App Runner	Container Apps	Cloud Run	26
Kubernetes	EKS	AKS	GKE	27
Object storage	S3	Blob Storage	Cloud Storage	28
Relational	RDS / Aurora	Azure SQL / PostgreSQL	Cloud SQL / AlloyDB	31
NoSQL	DynamoDB	Cosmos DB	Firestore	32
Queue / topic	SQS / SNS	Service Bus	Pub/Sub	24
Native IaC	CloudFormation / CDK	Bicep	Infrastructure Manager	34
Monitoring	CloudWatch	Azure Monitor	Cloud Monitoring	37
Generative AI	Bedrock	Microsoft Foundry	Agent Platform (Vertex AI)	40

LIMITS WORTH KNOWING

LIMIT	DEFAULT
Lambda timeout	15 min
Lambda sync payload	6 MB
SQS message	1 MiB
DynamoDB item	400 KB
S3 object	5 TB
CloudFormation resources / stack	500
Security group rules (in / out)	60 / 60
VPCs per Region	5 (adjustable)

PRICING MODELS

SERVICE	BILLED BY
EC2	Per second running + EBS + IPv4
Lambda	Requests + GB-seconds
Fargate	vCPU + GB per second
S3	GB-month + requests + egress
DynamoDB	Request units or provisioned capacity
NAT Gateway	Hourly + per GB
Bedrock	Tokens or provisioned units